CORVIT | DEVOPS ENGINEER EXPERT (AZ-400) | LOT 5

Capstone Class Lab

Ship it. Secure it. Scale it. Watch it. Clean it up.

One script per cloud for Azure and AWS free tier, run from the cloud shells, with a cleanup script.

Covers Days 1 to 6 of the course content

- Git and pull requests, Jenkins CI, Docker and registries, Kubernetes, Ansible and IaC, DevSecOps, monitoring
- AZ-400 and AZ-104 skills: identity, networking, compute, pipelines, security, instrumentation, cost
- Soft skills and freelancing: stakeholder update, change request, portfolio, proposal
- Trainer edition: step-by-step missions, answer key, assessment, troubleshooting, full scripts

At a glance



Item	Detail
What it is	One in-class capstone, run twice: once on Azure and once on AWS (or split the class). Each student runs ONE script per cloud from the cloud's own browser shell, and ONE cleanup script removes everything.
What students build	A small network, two servers (CI server with Jenkins and a private registry, plus a Kubernetes server running K3s), a secret held by a managed identity or role, a CPU alert, and a pipeline that tests, builds, scans, approves and publishes a container that is then rolled out, broken, healed and rolled back.
Course coverage	Every technical day and lab of the Corvit AZ-400 course content (Days 1 to 6): DevOps flow, Git and pull requests, Jenkins CI, Docker and registries, Kubernetes, Ansible and IaC, DevSecOps, monitoring, ethics and change management, freelancing deliverables.
Exam lens	AZ-400 (source control, pipelines, security, instrumentation) and AZ-104 (identity and governance, networking, compute, monitoring, cost).
Scripts (4 files)	capstone-azure.sh, cleanup-azure.sh, capstone-aws.sh, cleanup-aws.sh (full text in Appendices A to D, and delivered as separate files).
Duration	One training day, 09:00 to 17:00, including breaks. Cloud cost target: about zero (free tier), if cleanup is run the same day.

Contents

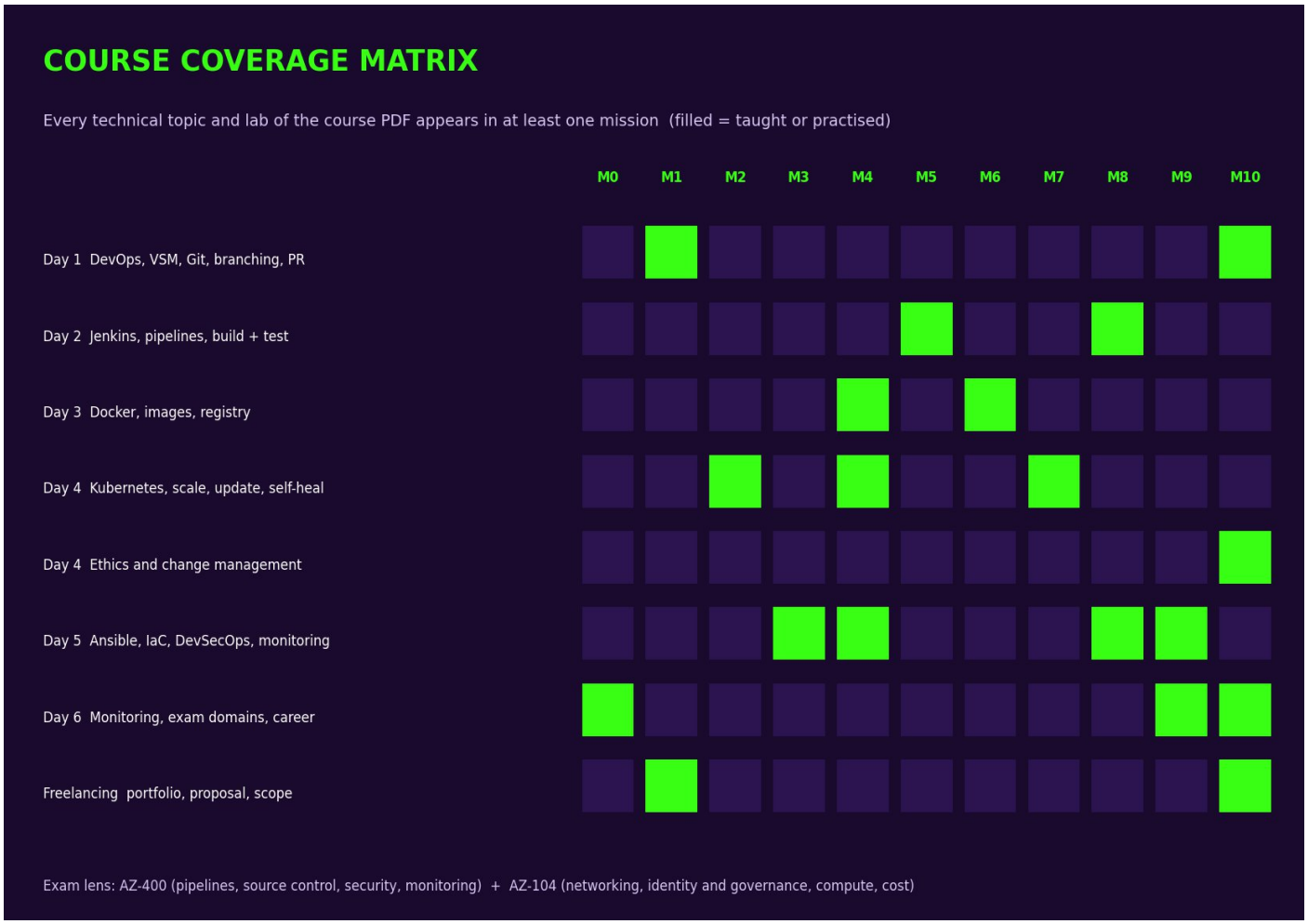
1	Purpose, outcomes and course coverage
2	Before you use this (honest status, prerequisites, trainer preparation)
3	Rules and free-tier guardrails
4	Class plan and how to split the cohort
5	Architecture and Azure to AWS service map
6	How the scripts work
7	The 11 missions, step by step (M0 to M10)
8	Professional deliverables (ethics, stakeholder, freelancing)
9	Assessment, trainer verification and answer key
10	Troubleshooting
A to D	Appendices: the four scripts, complete

1. Purpose, outcomes and course coverage

The course PDF (Lot 5, DevOps Engineer Expert AZ-400: Git, Jenkins, Docker, Kubernetes, Ansible) runs for 48 hours over six days. This capstone turns those days into one connected, script-driven exercise in which the student proves the whole delivery chain on a real cloud, then removes it again.

By the end of the day each student can:

- Create a Git repository, branch, commit, open and merge a pull request, and read a value stream (lead time, wait time, flow efficiency). (Day 1)
- Build a Jenkins declarative pipeline that tests, builds and publishes a container, and explain why a red test stops later stages. (Day 2)
- Write and read a Dockerfile, explain layers and caching, and push and pull from a private registry. (Day 3)
- Deploy to Kubernetes, scale, update with zero downtime, survive a bad release, roll back, and watch Kubernetes self-heal. (Day 4)
- Apply a configuration with an Ansible playbook, prove idempotency, and repair configuration drift. (Day 5)
- Apply least privilege: no SSH, source-restricted firewall rules, a managed identity or role, a secret store, a lock, and an image scan. (Day 5, AZ-400 security, AZ-104 identity)
- Create a metric alert, trigger it, and read the audit trail. (Days 5 and 6, AZ-400 instrumentation, AZ-104 monitoring)
- Write a stakeholder update, make and defend a change-request decision, and package the work as a portfolio case study and freelance offer. (Days 1, 2, 3, 4, 5, 6 soft and freelancing skills)



Course PDF topic to mission map

Course PDF topic	Mission	Script phase or tool
DevOps culture, lifecycle, value stream mapping (Day 1)	M1, M10	report (lead time, flow efficiency)
Intro to Azure DevOps services; GitHub / Azure Repos workflows (Day 1)	M1, M5	GitHub repo and PR; Azure Pipelines is mapped on paper
Git: repositories, commits, branches, merges, branching strategy (Day 1)	M1	export, then git in the shell
Communicating pipeline outcomes; documenting CI/CD (Day 1 soft skill)	M10, Section 8	Stakeholder message, README
CI concepts, Jenkins architecture and installation (Day 2)	M4, M5	compute installs Jenkins
Declarative pipelines, build automation, test integration, Git with Jenkins (Day 2)	M5	Jenkinsfile in the repo
Freelancing introduction, portfolio, profile (Day 2)	Section 8	Case study and profile text
Containerization, Docker architecture, images, containers, registries (Day 3)	M6	Registry on :5000, docker history
Dockerfiles and container networking (Day 3)	M6, M7	Dockerfile, NodePort 30080
Azure Container Registry (Day 3)	M6	Paper mapping (ACR Basic is not free)
Freelance project management, proposals (Day 3)	Section 8	Proposal template
Kubernetes architecture, pods, services, deployments (Day 4)	M7	kube helper, K3s
AKS; scaling, rolling updates, self-healing (Day 4)	M7	K3s on both clouds; AKS mapped on paper
DevOps ethics, access control, change management (Day 4 soft skill)	M3, M10	Least privilege; Friday 4:40 pm decision
Configuration management, Ansible, IaC (Day 5)	M4, M8	Baseline playbook, drift repair
DevSecOps and security in CI/CD (Day 5)	M3, M8	Trivy scan, approval gate, secret store
Monitoring and logging pipelines (Days 5 and 6)	M9	Alert, activity log, service logs
Freelancing business development: pricing, scope, branding (Day 5)	Section 8	Package and price table
AZ-400 domains, mock exam, troubleshooting (Day 6)	M0 to M10, Section 9	Check questions, answer key
Resume, interview, portfolio, GitHub presence (Day 6 soft skill)	Section 8	Case study, README, pinned repo

What is deliberately NOT covered (be honest with students)

Not free, so paper-only: Azure Container Registry, AKS, Azure Pipelines hosted parallelism, Application Insights dashboards. Students map them in the check questions. K3s stands in for AKS on both clouds.

Thin coverage: Ansible is one playbook (no inventory, roles or templates); IaC is a Bash script plus Ansible, not Bicep, Terraform or CloudFormation; AZ-104 storage is not covered at all.

Use the optional stretch ideas in Section 9 if you want to extend any of these.

2. Before you use this

Honest status of the scripts

The four scripts passed bash -n and ShellCheck. The app, tests, Kubernetes YAML and Ansible playbook were unpacked from the generated user-data and tested offline (2 tests pass; YAML and ansible-playbook --syntax-check pass; user-data is about 5 KB, far below the 16 KB AWS and 64 KB Azure limits).

They have NOT been run against live Azure or AWS accounts by the author. Cloud CLIs, free-tier rules, VM size availability and package URLs change. A trainer dry run the day before is required (checklist below).

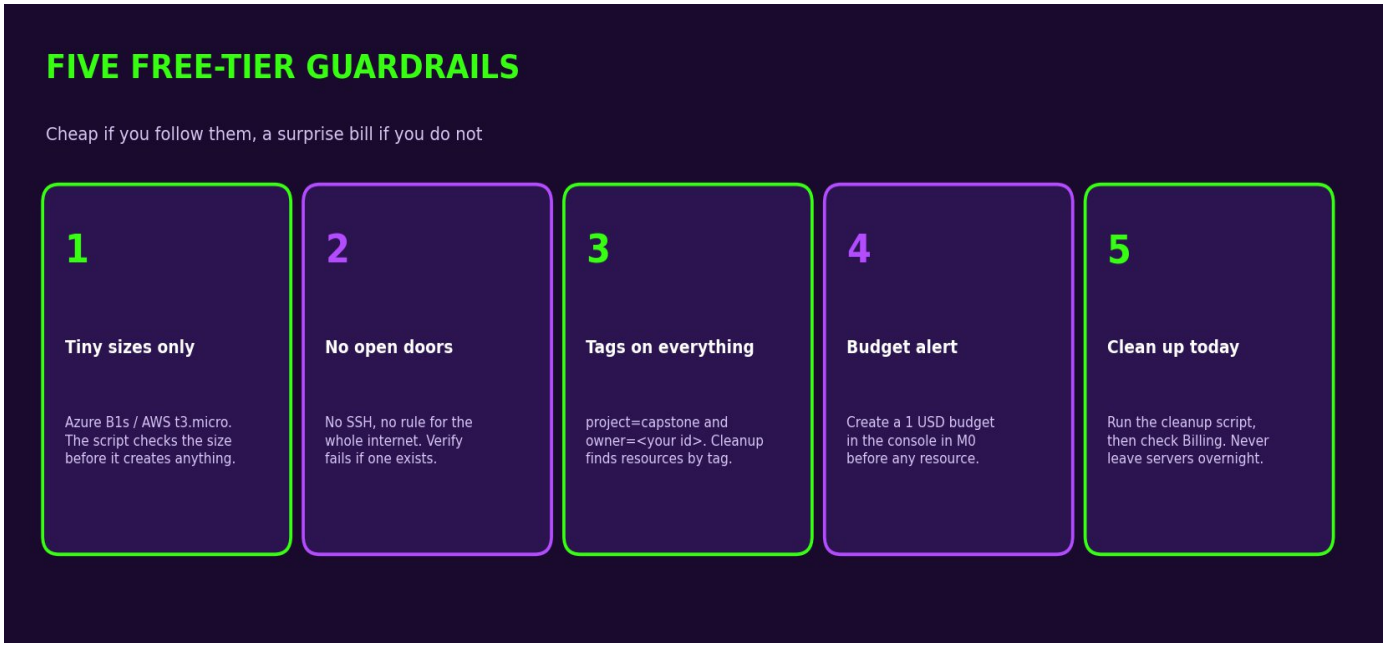
Trainer preparation checklist (do this the day before)

#	Task	Why
1	Run one full cycle yourself on each cloud: all, M5 build, M7 rollout, verify, cleanup. Note the time of each phase.	Finds any CLI or region problem while it is cheap to fix.
2	Confirm every student can sign in to an Azure free account and an AWS account (both normally need a card and a phone number at sign-up). Plan a fallback: trainer-owned sandbox accounts, each student limited to their own STUDENT_ID.	Sign-up is the most common blocker; it can eat the first hour.
3	Azure: confirm Standard_B1s (or B2ats_v2) is offered in your chosen LOCATION; AWS: run check to see the free-tier eligible instance types of the account type (older accounts: t2/t3.micro; accounts created after 15 July 2025 are on the newer free plan with credits and other eligible types).	The check phase refuses to continue on a size that is unavailable or not free.
4	Check the Azure account may create role assignments (Owner on the subscription) and AWS may create IAM roles. Students on restricted "student" subscriptions may fail in M3.	M3 needs RBAC and IAM rights.
5	Verify the Jenkins apt key URL (jenkins.io-2026.key) and the Ubuntu 24.04 image names still resolve.	Both are inside the bootstrap; a change breaks the CI server.
6	Decide Track A (Azure) or Track B (AWS) per student, or both for fast groups (Section 4).	Time and cost control.
7	Prepare a shared sheet for STUDENT_ID, score, token and cleanup status.	Live monitoring and marking.

Student prerequisites

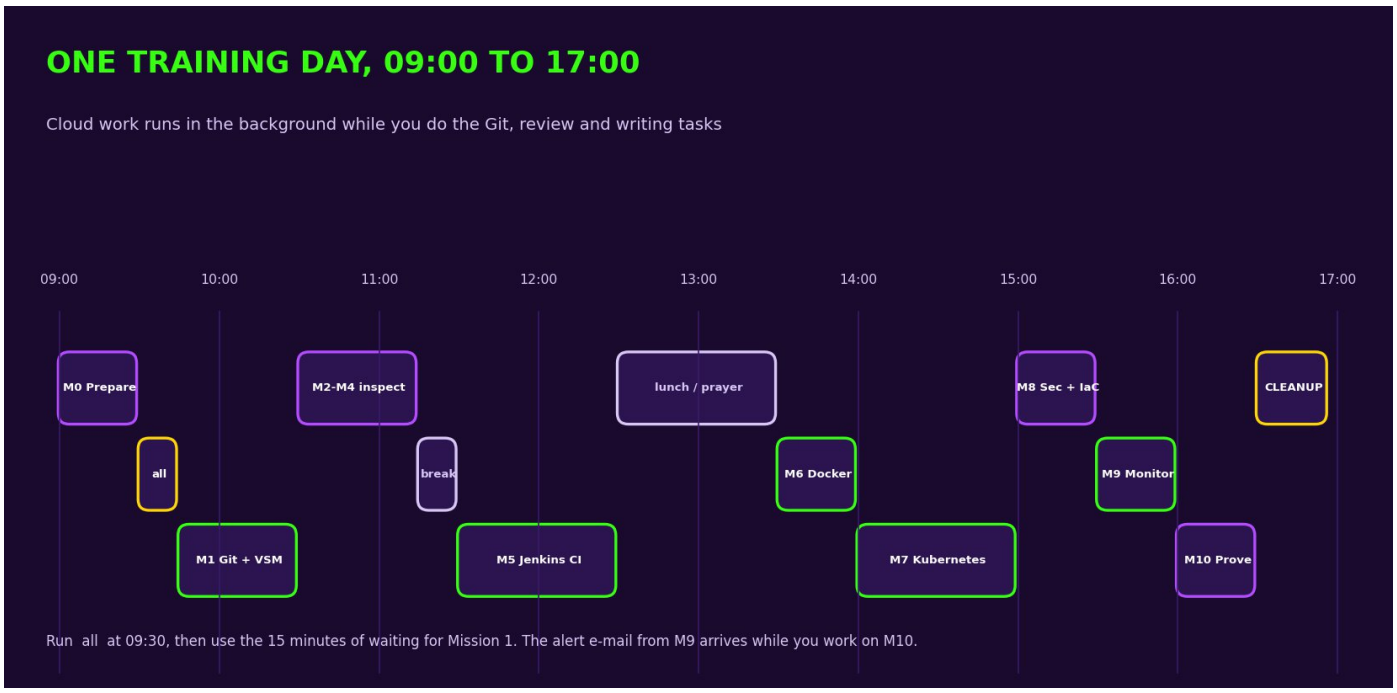
- A free Azure account and/or an AWS account, a GitHub account, a laptop with a modern browser, and a mailbox they can open during class.
- Days 1 to 5 knowledge or the workbooks. Nothing needs to be installed locally: everything runs in the cloud shells.
- Their own public IP address (open https://ifconfig.me in the laptop browser; it changes with VPN or mobile data).

3. Rules and free-tier guardrails



- **One student, one STUDENT_ID** (3 to 8 lowercase letters or digits). All names, tags and the Key Vault or parameter path derive from it, so many students can share one subscription without collisions.
- **No server is ever left overnight.** Cleanup is a graded step (5 marks).
- **No secrets in Git, chat or screenshots.** The GitHub token, the Jenkins unlock password and the secret value stay out of evidence (the secret is only shown as a length).
- **Your own work.** Evidence must show your STUDENT_ID and the token printed by verify. Answers that only repeat documentation score at most 3 of 10 (same rule as the Day 4 and 5 descriptive questions).
- **Budgets alert, they do not stop spending.** Azure and AWS budgets send e-mails; only deletion stops cost.
- **Free terms change.** Public IPv4 addresses and disks can bill a few cents per hour even on free plans. Check the provider's current free-tier page the same morning.

4. Class plan and how to split the cohort



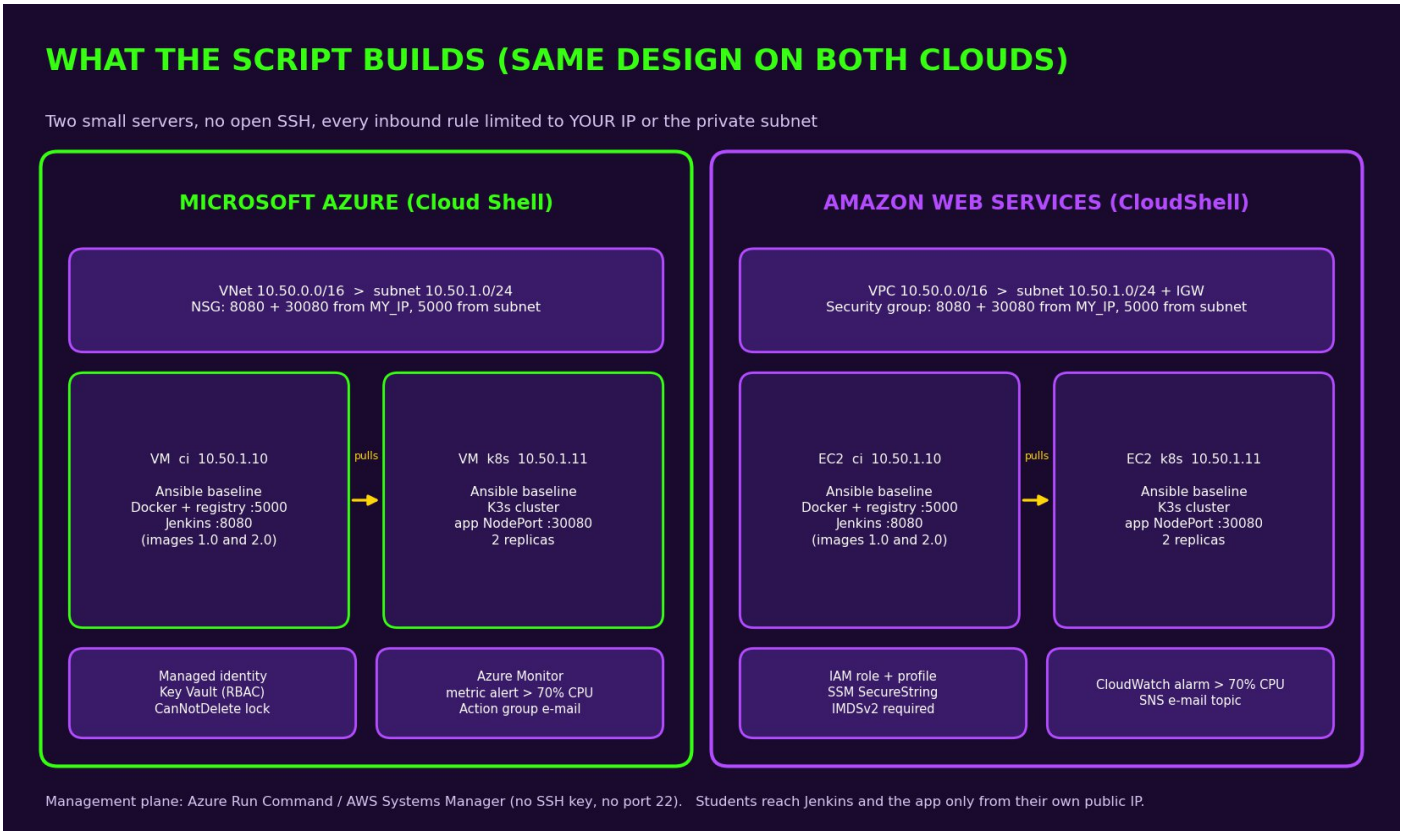
Option	How it works	Best when
Track split	Half the class does Azure, half does AWS. In M10 pairs (one Azure, one AWS) compare their report and verify output and fill the comparison table in Section 8.	Time is tight, accounts are limited. Recommended.
Both clouds	Each student runs both scripts. Run Azure all and AWS all in two Cloud Shell windows at 09:30; do missions on one cloud and repeat only M2, M3, M9 on the other.	Strong group, two free accounts per student.
Instructor demo + student run	Trainer shows the other cloud on the projector while students run their own.	Mixed ability.

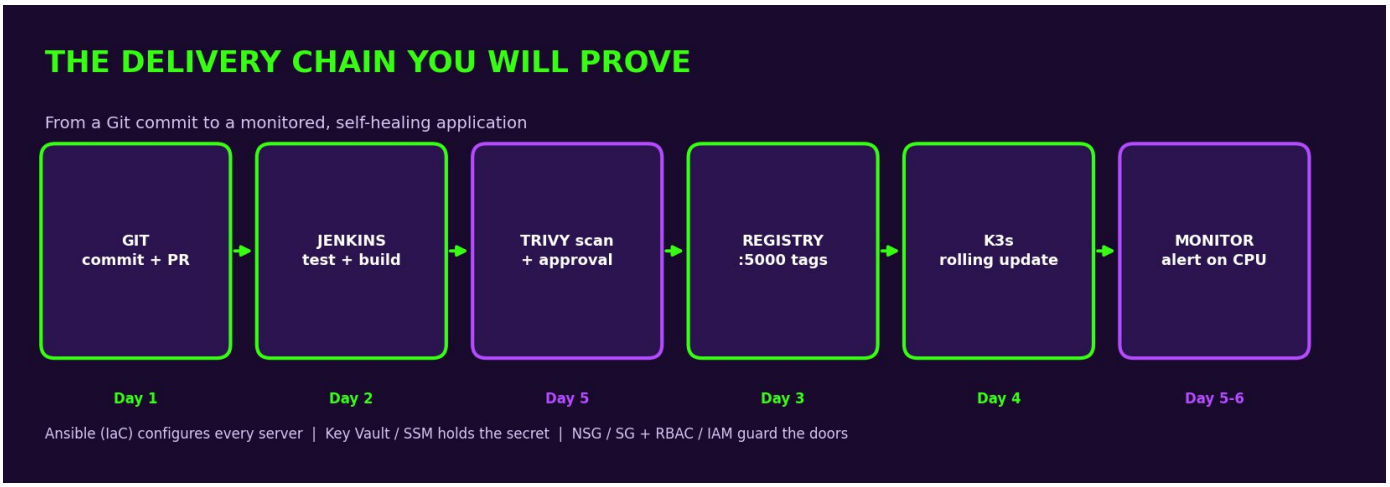
Cloud Shell hygiene

Azure Cloud Shell and AWS CloudShell both disconnect after about 20 minutes idle; running scripts stop. If that happens, re-export your variables and run only the phases that did not finish (Section 6).

Azure Cloud Shell creates a small storage account in cloud-shell-storage-* the first time. It is outside this lab and is NOT removed by cleanup; delete it at the end of the course.

5. Architecture and Azure to AWS service map





Concept	Azure	AWS	Used in
Browser shell	Cloud Shell (Bash)	CloudShell	M0
Grouping and tags	Resource group, tags	Tags (+ Resource Groups)	M2, M10
Network	VNet, subnet	VPC, subnet, internet gateway, route table	M2
Firewall	Network security group (subnet)	Security group (instance)	M2
Server	Virtual machine (B1s)	EC2 instance (t3.micro)	M4
Management without SSH	Run Command	Systems Manager Run Command	M2 to M9

Concept	Azure	AWS	Used in
Workload identity	System-assigned managed identity + RBAC role	IAM role + instance profile	M3
Secret store	Key Vault (RBAC mode)	SSM Parameter Store SecureString	M3
Protect from deletion	Resource lock (CanNotDelete)	Closest: termination / deletion protection, SCP	M3
Guardrail policy	Azure Policy (Allowed locations)	Closest: SCP, AWS Config rule	M3
Metadata hardening	Managed identity endpoint	IMDSv2 required	M3
Alerting	Metric alert + action group	CloudWatch alarm + SNS topic	M9
Audit trail	Activity log	CloudTrail event history	M9
Cost control	Cost Management budgets	AWS Budgets	M0, M10
Pipelines (concept)	Azure Pipelines YAML	Jenkins here; CodePipeline as alternative	M5 questions
Registry (paper)	Azure Container Registry	Elastic Container Registry	M6
Managed Kubernetes (paper)	AKS	EKS	M7
Declarative IaC (paper)	Bicep / ARM / Terraform	CloudFormation / Terraform	M8

6. How the scripts work

Each cloud has one provisioning script with named phases and one cleanup script. A phase is a function you can run alone: ./capstone-azure.sh network. The script keeps a small state file and a timing log in ~/capstone-<cloud>-<id>/.

Phase	Azure creates	AWS creates	Time
check	Nothing. Login, region, VM size, IP format.	Nothing. Login, free-tier eligibility, IP format.	1 min
network	Resource group, VNet, subnet, NSG with 3 rules	VPC, subnet, IGW, route table, security group with 3 rules	2 min
identity	Key Vault, secret, 2 role assignments, lock, policy (runs after compute)	IAM role and profile, SecureString (runs before compute)	2 min
compute	Two Ubuntu 24.04 VMs with managed identity, user-data bootstrap	Two EC2 instances, IMDSv2, no key pair, user-data bootstrap	4 min
wait	Polls until both servers finish installing	Same, plus waits for the SSM agent	8 to 12 min
monitor	Action group + CPU metric alert	SNS topic + CloudWatch alarm	1 min
status	Prints URLs and portal hints	Prints URLs and console hints	seconds
export	Writes a Git-ready copy of the app to ~/capstone-app-<id>	Same	seconds
run, kube	Run a command on a server through Run Command	Same through Systems Manager	seconds each
secret	Read or try to write the secret with the VM identity	Same with the instance role	seconds
ansible, load	Re-run the playbook; start 6 minutes of CPU load	Same	seconds
verify, report	13 automatic checks and a score; value-stream table	15 automatic checks and a score; value-stream table	2 min

Variables (set before the first phase)

Variable	Meaning	Example
STUDENT_ID	Required. 3 to 8 lowercase letters or digits.	export STUDENT_ID=amk07
MY_IP	Required. Your public IP from your own browser. Used in every inbound rule as /32.	export MY_IP=203.0.113.7
EMAIL	Needed by monitor. Receives the alert.	export EMAIL=you@example.com

Variable	Meaning	Example
LOCATION / REGION	Azure region (default southeastasia) / AWS region (default ap-south-1).	export REGION=ap-south-1
VM_SIZE / INSTANCE_TYPE	Server size. Defaults are the smallest free sizes. Larger sizes may cost money.	export INSTANCE_TYPE=t3.micro

If `all` stops halfway

Re-export the variables, then run the missing phases in order. **Azure:** check, network, compute, wait, identity, monitor, status. **AWS:** check, network, identity, compute, wait, monitor, status.

Never repeat network or compute once they succeeded: the second run creates duplicates or fails. If a phase failed halfway, run the cleanup script and start again.

What the servers install by themselves (bootstrap)

Both: Ansible baseline playbook (2 GB swap, tools, marker file) run in local mode. **ci server:** Docker, a private registry on port 5000 that already holds the app images 1.0 and 2.0, then Jenkins with a 512 MB heap and the jenkins user in the docker group. **k8s server:** K3s (Traefik off to save memory) trusting the registry at 10.50.1.10:5000. The full log is in /var/log/capstone-bootstrap.log.

7. The 11 missions, step by step

Read each mission table left to right. Where Azure and AWS differ, two columns are shown; where they are the same, one wide cell is shown. Commands starting with ./capstone- are typed in the cloud shell. <id> means your STUDENT_ID. Wait for each command to finish before the next.

> **Shortcut used in many steps**
> run ci "cmd" or run k8s "cmd" runs a Linux command on that server; kube <args> runs kubectl <args> on the k8s server. Nobody opens an SSH port.

M0 Prepare, guardrails and a dry check

Course link	Day 6 (readiness); AZ-104 cost management
Time	30 min (09:00 to 09:30)
Exam skill	AZ-104: monitor and maintain, cost governance

Nothing is created in this mission except a budget. You are proving that you can reach the cloud shell and that the plan fits your account.

#	What you do	Azure (Cloud Shell and portal)	AWS (CloudShell and console)
1	Open the browser shell.	portal.azure.com, click the >_ icon, choose Bash. Accept the small storage account on first use.	console.aws.amazon.com, click the CloudShell icon in the top bar. Confirm the region at top right (ap-south-1).
2	Create a budget BEFORE anything else.	Cost Management + Billing > Budgets > Add. 1 USD monthly, alert at 80 percent to your e-mail.	Billing and Cost Management > Budgets > Create > Zero spend budget template.
3	Upload the two scripts for your cloud.	Cloud Shell toolbar > Manage files > Upload: capstone-azure.sh, cleanup-azure.sh.	CloudShell > Actions > Upload file: capstone-aws.sh, cleanup-aws.sh.
4	Make them runnable.	chmod +x *.sh	chmod +x *.sh
5	Find your public IP in YOUR laptop browser (not in the shell).	Open https://ifconfig.me	Open https://ifconfig.me
6	Set your variables.	export STUDENT_ID=amk07 MY_IP=203.0.113.7 EMAIL=you@example.com	export STUDENT_ID=amk07 MY_IP=203.0.113.7 EMAIL=you@example.com
7	Dry check.	./capstone-azure.sh check	./capstone-aws.sh check
8	Read the help once.	./capstone-azure.sh help	./capstone-aws.sh help

> **Evidence to keep (screenshots, file names M#_what.png)**
> M0_budget.png (budget page), M0_check.png (check output with [OK] lines).

Check question (answer in your own words, using your own output)
A budget at 80 percent alerts you. Does it stop the spending? What actually stops it? (Expected: no; deleting the resources.)

M1 Source control and the value stream

Course link	Day 1: Git, branches, pull requests, value stream mapping
Time	45 min (09:45 to 10:30), while all builds the cloud
Exam skill	AZ-400: source control strategy; processes and communications

Start the build first, then use the 15 minutes of waiting for Git. The cloud is being created in tab 1 while you work in tab 2.

#	What you do	Azure (Cloud Shell and portal)	AWS (CloudShell and console)
1	Start the whole build in the first shell tab. Leave it running (about 15 minutes).	./capstone-azure.sh all	./capstone-aws.sh all
2	Open a SECOND shell tab or session and re-export the same three variables.	Cloud Shell > Open new session.	CloudShell > Actions > New tab.
3	Write the app to a Git-ready folder.	./capstone-azure.sh export	./capstone-aws.sh export
4	On github.com create an EMPTY public repository named capstone-app (no README).	GitHub > New repository.	
5	Make three meaningful commits.	cd ~/capstone-app-$STUDENT_ID git init -b main git config user.name "Your Name" git config user.email "you@example.com" git add README.md .gitignore && git commit -m "docs: describe the project" git add app tests requirements*.txt && git commit -m "feat: flask app and tests" git add Dockerfile .dockerignore Jenkinsfile k8s ansible bootstrap.sh && git commit -m "ci: pipeline, container, deployment"	
6	Push to YOUR repository. The password is a GitHub personal access token (classic, scope repo, 7 days). Never	git remote add origin https://github.com/<user>/capstone-app.git git push -u origin main	

#	What you do	Azure (Cloud Shell and portal)	AWS (CloudShell and console)
	paste it into a file or screenshot.		
7	Branch, change, pull request, review, squash merge.	git switch -c feat/greeting sed -i 's/Capstone app/Capstone app (class demo)/' app/main.py git commit -am "feat: label the demo" git push -u origin feat/greeting GitHub: open the pull request, add one review comment, Squash and merge.	
8	Value stream, part 1: write your honest estimate of doing this whole environment by clicking in the console (process time, waiting time, hours).	Worksheet line: "Manual estimate: PT ___ h, WT ___ h". Compare in M10.	

Evidence to keep (screenshots, file names M#_what.png)

M1_git_log.png (git log --oneline --graph), M1_pr.png (merged pull request showing your username), your manual estimate.

Check question (answer in your own words, using your own output)

Why is a pull request with a review a control, not just a habit? Name the Azure Repos feature that enforces it. (Expected: branch policy with required reviewer and build validation.)

M2 Network and least privilege

Course link	Day 3 and 4 container networking; Day 5 security
Time	20 min (10:30 to 10:50)
Exam skill	AZ-104: configure and manage virtual networking

By now tab 1 should have printed the URLs. If it is still running, wait for status.

#	What you do	Azure (Cloud Shell and portal)	AWS (CloudShell and console)
1	Print the URLs.	./capstone-azure.sh status	./capstone-aws.sh status
2	Open the firewall rules in the console.	Resource groups > rg-cap-<id> > nsg-cap-<id> > Inbound security rules.	VPC > Security groups > cap-<id>-sg > Inbound rules.
3	List the three rules (port, source). Note the source of 8080 and 30080 is your IP /32, and that there is	Write the table in your notes.	

#	What you do	Azure (Cloud Shell and portal)	AWS (CloudShell and console)
	no port 22 rule.		
4	Prove the restriction works. From the SHELL (a different IP than your laptop) try Jenkins; it must time out. Then open the same URL from your LAPTOP; it must load.	curl -m 5 http://<ci public ip>:8080 (expect a timeout)	
5	Find where each address is used: private IPs 10.50.1.10 (ci) and 10.50.1.11 (k8s), port 5000, port 30080.	Answer in your notes: which component uses each value?	

Evidence to keep (screenshots, file names M#_what.png)
M2_rules.png (rules with your IP), M2_timeout.png (curl timeout in the shell).

Check question (answer in your own words, using your own output)
Why is port 5000 open only to the subnet and why is there no SSH rule? Name the Azure features that replace an open SSH port. (Expected: Bastion, just-in-time access, Run Command; AWS: Session Manager.)

M3 Identity, secrets and governance

Course link	Day 4 ethics and access control; Day 5 DevSecOps
Time	25 min (10:50 to 11:15)
Exam skill	AZ-104: identities and governance; AZ-400: security and compliance

The servers hold no password and no access key. Identity is what lets the CI server read one secret.

#	What you do	Azure (Cloud Shell and portal)	AWS (CloudShell and console)
1	Show who may do what.	Key vaults > kv<id>... > Access control (IAM) > Role assignments. Find your account (Secrets Officer) and the VM (Secrets User).	IAM > Roles > cap-<id>-ec2-role > Permissions: SSM core policy and the inline policy read-own-params.
2	Show the server identity.	VM cap-<id>-ci > Identity > System assigned = On.	EC2 > cap-<id>-ci > Security tab > IAM role; Details > Metadata version: V2 only.
3	Read the secret as the	./capstone-azure.sh secret	./capstone-aws.sh secret

#	What you do	Azure (Cloud Shell and portal)	AWS (CloudShell and console)
	server does. Only the length is shown.		
4	Try to WRITE with the read-only identity. It must be refused.	./capstone-azure.sh secret write	./capstone-aws.sh secret write
5	Try to delete the secret store.	Try deleting the Key Vault in the portal. The lock blocks it (even for Owner).	EC2: note that nothing stops you, so for production you would enable termination protection. Write that in your notes.
6	Optional: guardrail policy.	Resource group > Policies > Assignments: "Allowed locations (capstone)".	Read about service control policies (organization level); no action.

Evidence to keep (screenshots, file names M#_what.png)
M3_iam.png, M3_read.png (secret length), M3_write_denied.png (HTTP 403 or AccessDenied), M3_lock.png (portal error).

Check question (answer in your own words, using your own output)
Why Secrets User and not Owner, and why scope the role to the vault only? What does a lock protect against that RBAC does not? (Expected: least privilege and blast radius; accidental deletion by someone who has rights.)

M4 Compute and the Ansible baseline

Course link	Day 5: Ansible, IaC; Day 2: Jenkins installation
Time	25 min (up to 11:15 with M3)
Exam skill	AZ-104: deploy and manage compute resources

The servers configured themselves. You now check how, and you break the configuration on purpose to see Ansible repair it.

#	What you do	Azure (Cloud Shell and portal)	AWS (CloudShell and console)
1	Inspect the two servers: size, disk, private and public IP, tags.	Virtual machines > cap-<id>-ci and -k8s > Overview, Disks, Tags.	EC2 > Instances > filter tag owner=<id> > Details, Storage, Tags.
2	Read the end of the bootstrap log.	./capstone-azure.sh run ci 'tail -n 8 /var/log/capstone-bootstrap.log'	./capstone-aws.sh run ci 'tail -n 8 /var/log/capstone-bootstrap.log'
3	Show the baseline: marker file and swap.	./capstone-azure.sh run k8s 'cat /etc/capstone-baseline.txt; free -h | head -n 3'	./capstone-aws.sh run k8s 'cat /etc/capstone-baseline.txt; free -h | head -n 3'

#	What you do	Azure (Cloud Shell and portal)	AWS (CloudShell and console)
4	Run the playbook a second time. Read the PLAY RECAP: it must say changed=0.	./capstone-azure.sh ansible k8s	./capstone-aws.sh ansible k8s
5	Create drift: switch swap off by hand, confirm, then let Ansible repair it.	./capstone-azure.sh run k8s 'swapoff /swapfile; swapon --show | wc -l' ./capstone-azure.sh ansible k8s	./capstone-aws.sh run k8s 'swapoff /swapfile; swapon --show | wc -l' ./capstone-aws.sh ansible k8s
6	Confirm repair.	./capstone-azure.sh run k8s 'free -h | grep -i swap'	./capstone-aws.sh run k8s 'free -h | grep -i swap'

Evidence to keep (screenshots, file names M#_what.png)
M4_instances.png, M4_changed0.png (recap with changed=0), M4_drift.png (recap showing changed=1 then repaired).

Check question (answer in your own words, using your own output)
What does changed=0 prove, and why is that the point of configuration management? Name the Azure or AWS declarative tools that do the same job for the infrastructure itself. (Expected: idempotence, desired state; Bicep/ARM/Terraform, CloudFormation.)

M5 Continuous integration with Jenkins

Course link	Day 2: Jenkins pipelines, build, test, Git integration
Time	60 min (11:30 to 12:30)
Exam skill	AZ-400: build and release pipelines

Jenkins on a 1 GB server is slow the first time (plugins, Trivy database). Be patient and do not start two builds at once.

#	What you do	Azure (Cloud Shell and portal)	AWS (CloudShell and console)
1	Open Jenkins from your laptop. It is plain HTTP and only your IP can reach it.	Browser: http://<ci public ip>:8080	
2	Get the unlock password and finish the setup wizard (suggested plugins, create admin user).	./capstone-azure.sh run ci 'cat /var/lib/jenkins/secrets/initialAdminPassword'	./capstone-aws.sh run ci 'cat /var/lib/jenkins/secrets/initialAdminPassword'
3	New Item > Pipeline named capstone-pipeline > Definition: Pipeline script from	Jenkins UI	

#	What you do	Azure (Cloud Shell and portal)	AWS (CloudShell and console)
	SCM > Git > your repo URL > branch */main > Script Path Jenkinsfile.		
4	Build Now. Watch the stages: Checkout, Test, Build image, Scan image. It then PAUSES at Approval. Click Proceed in the stage view (this is your approval gate).	Jenkins UI. First build: 5 to 10 minutes.	
5	Prove the image was published with the build number as its tag.	./capstone-azure.sh run ci 'curl -s localhost:5000/v2/capstone-app/tags/list'	./capstone-aws.sh run ci 'curl -s localhost:5000/v2/capstone-app/tags/list'
6	Break the build on purpose. In GitHub create branch fix/break, edit tests/test_app.py so the health test expects b"fine" instead of b"ok", open a PR, merge, Build Now.	GitHub + Jenkins UI	
7	Observe a RED build: note the exact failing line, the build number, and which stages were skipped.	Console Output and Test Result	
8	Fix it by reverting the merge in GitHub (Revert button on the merged PR), merge, build again: GREEN.	GitHub + Jenkins UI	

Evidence to keep (screenshots, file names M#_what.png)

M5_stage_view.png (green run with the approval step), M5_red.png (red run with build number and failing line), M5_tags.png (tags list).

Check question (answer in your own words, using your own output)
What happened to the stages after Test when it failed, and how did you confirm the registry did not get a new tag? How would you get the same behaviour in an Azure Pipelines YAML file? (Expected: later stages skipped; stages, dependsOn, conditions, branch policy with build validation, environment approval.)

M6 Containers and the private registry

Course link	Day 3: Docker images, Dockerfile, registry, ACR
Time	30 min (13:30 to 14:00)
Exam skill	AZ-400: build optimization, container images

#	What you do	Azure (Cloud Shell and portal)	AWS (CloudShell and console)
1	Images and the registry container.	./capstone-azure.sh run ci 'docker images; docker ps --format "table {{.Names}}\t{{.Status}}"'	./capstone-aws.sh run ci 'docker images; docker ps --format "table {{.Names}}\t{{.Status}}"'
2	Show the layers of the app image (first lines).	./capstone-azure.sh run ci 'docker history localhost:5000/capstone-app:1.0 | head -n 8'	./capstone-aws.sh run ci 'docker history localhost:5000/capstone-app:1.0 | head -n 8'
3	Who does the container run as?	./capstone-azure.sh run ci 'docker run --rm localhost:5000/capstone-app:1.0 whoami'	./capstone-aws.sh run ci 'docker run --rm localhost:5000/capstone-app:1.0 whoami'
4	Prove layer caching. Append a comment line to the app, rebuild, and read which steps used the cache; then undo the change.	./capstone-azure.sh run ci 'cd /opt/capstone && echo "# change" >> app/main.py && docker build -t cache-test . 2>&1 | grep -iE "cache|COPY|RUN|Step" | head -n 14; sed -i "\$d" app/main.py'	./capstone-aws.sh run ci 'cd /opt/capstone && echo "# change" >> app/main.py && docker build -t cache-test . 2>&1 | grep -iE "cache|COPY|RUN|Step" | head -n 14; sed -i "\$d" app/main.py'
5	List what the registry holds.	./capstone-azure.sh run ci 'curl -s localhost:5000/v2/_catalog'	./capstone-aws.sh run ci 'curl -s localhost:5000/v2/_catalog'
6	Paper exercise: write the two commands you would use to create a registry and push this image to ACR (Azure) or ECR (AWS). Do not create it (not free).	Notes	

Evidence to keep (screenshots, file names M#_what.png)
M6_history.png, M6_whoami.png, M6_cache.png (cache lines).

Check question (answer in your own words, using your own output)
Which step was rebuilt after your code change and which used the cache? Which single change in the Dockerfile order would make every build slow? How does this affect cost on a hosted pipeline agent? (Expected: COPY app after pip install keeps dependencies cached; copying the code before installing breaks the cache; build minutes cost money.)

M7 Kubernetes: scale, update, fail, heal, roll back

Course link	Day 4: Pods, services, deployments, scaling, rolling updates, self-healing
Time	60 min (14:00 to 15:00)
Exam skill	AZ-400: deployment strategies, release gates and rollback

Use a second shell tab for the live request loop in step 6. Keep the first tab for commands.

#	What you do	Azure (Cloud Shell and portal)	AWS (CloudShell and console)
1	Deploy the app.	./capstone-azure.sh kube apply -f /opt/capstone/k8s/deployment.yaml	./capstone-aws.sh kube apply -f /opt/capstone/k8s/deployment.yaml
2	Wait for 2 pods Running (the first pull from the registry takes a minute). Note pod names and IPs.	./capstone-azure.sh kube get pods -o wide	./capstone-aws.sh kube get pods -o wide
3	Open the app from your laptop. You must see your id and your token.	Browser: http://<k8s public ip>:30080	
4	See load balancing: ten requests, two different pod names.	./capstone-azure.sh run k8s 'for i in $(seq 1 10); do curl -s localhost:30080/host; done'	./capstone-aws.sh run k8s 'for i in $(seq 1 10); do curl -s localhost:30080/host; done'
5	Scale up and back down.	./capstone-azure.sh kube scale deployment capstone-app --replicas=3 ./capstone-azure.sh kube get pods ./capstone-azure.sh kube scale deployment capstone-app --replicas=2	./capstone-aws.sh kube scale deployment capstone-app --replicas=3 ./capstone-aws.sh kube get pods ./capstone-aws.sh kube scale deployment capstone-app --replicas=2
6	ZERO-DOWNTIME UPDATE. Tab 2: start a request loop. Tab 1:	Tab 2: ./capstone-azure.sh run k8s 'for i in $(seq 1 40); do curl -s -o	Tab 2: ./capstone-aws.sh run k8s 'for i in $(seq 1 40); do curl -s -o /dev/null -

#	What you do	Azure (Cloud Shell and portal)	AWS (CloudShell and console)
	update to version 2.0 and watch the rollout. Every response in tab 2 must be 200.	/dev/null -w "%{http_code} " localhost:30080/; sleep 0.5; done' Tab 1: ./capstone-azure.sh kube set image deployment/capstone-app app=10.50.1.10:5000/capstone-app:2.0 ./capstone-azure.sh kube rollout status deployment/capstone-app	w "%{http_code} " localhost:30080/; sleep 0.5; done' Tab 1: ./capstone-aws.sh kube set image deployment/capstone-app app=10.50.1.10:5000/capstone-app:2.0 ./capstone-aws.sh kube rollout status deployment/capstone-app
7	FAILED RELEASE. Point at a tag that does not exist, wait 25 seconds, capture the pods and the event, and show that users still get answers.	./capstone-azure.sh kube set image deployment/capstone-app app=10.50.1.10:5000/capstone-app:9.9 ./capstone-azure.sh run k8s 'sleep 25; kubectl get pods; kubectl describe pods | grep -iE "Failed|BackOff" | head -n 4; curl -s localhost:30080/'	./capstone-aws.sh kube set image deployment/capstone-app app=10.50.1.10:5000/capstone-app:9.9 ./capstone-aws.sh run k8s 'sleep 25; kubectl get pods; kubectl describe pods | grep -iE "Failed|BackOff" | head -n 4; curl -s localhost:30080/'
8	ROLL BACK and confirm the image.	./capstone-azure.sh kube rollout undo deployment/capstone-app ./capstone-azure.sh kube rollout status deployment/capstone-app	./capstone-aws.sh kube rollout undo deployment/capstone-app ./capstone-aws.sh kube rollout status deployment/capstone-app
9	SELF-HEALING. Delete one pod and watch Kubernetes replace it.	./capstone-azure.sh run k8s 'kubectl delete pod $(kubectl get pods -o name | head -n1); sleep 6; kubectl get pods'	./capstone-aws.sh run k8s 'kubectl delete pod $(kubectl get pods -o name | head -n1); sleep 6; kubectl get pods'
10	Close the CI to CD loop: deploy the image YOUR Jenkins built (the tag is the build number from M5). The page now shows that version.	./capstone-azure.sh kube set image deployment/capstone-app app=10.50.1.10:5000/capstone-app:1	./capstone-aws.sh kube set image deployment/capstone-app app=10.50.1.10:5000/capstone-app:1
11	A Kubernetes Secret is only encoded. Create one and decode it.	./capstone-azure.sh run k8s 'kubectl create secret generic demo --from-literal=API_KEY=demo12345; kubectl get secret demo -o jsonpath="{.data.API_KEY}" | base64 -d; echo'	./capstone-aws.sh run k8s 'kubectl create secret generic demo --from-literal=API_KEY=demo12345; kubectl get secret demo -o jsonpath="{.data.API_KEY}" | base64 -d; echo'

Evidence to keep (screenshots, file names M#_what.png)
M7_pods_running.png, M7_browser.png (id and token), M7_loop_200.png, M7_pull_error.png (pods + event for 9.9), M7_rollback.png, M7_heal.png, M7_secret_decoded.png.

Check question (answer in your own words, using your own output)
Using the two settings in your own deployment.yaml (maxUnavailable and the readiness probe), why did users still get answers during the 9.9 failure? Which gate or check would stop a bad release in Azure DevOps and which action rolls it back? (Expected: maxUnavailable 0 and readiness keep old pods serving; environment approval or check and automated rollback / rollout undo.)

M8 DevSecOps and infrastructure as code

Course link	Day 5: DevSecOps, IaC; Day 2: pipeline stages
Time	30 min (15:00 to 15:30)
Exam skill	AZ-400: security and compliance; AZ-104: governance

#	What you do	Azure (Cloud Shell and portal)	AWS (CloudShell and console)
1	Read the Scan image stage of your green build in the Jenkins console. How many CRITICAL findings did it report? Why does the build still pass?	Jenkins Console Output. The Jenkinsfile uses --exit-code 0, so it only reports.	
2	See a scan gate that FAILS: scan an old image on purpose and read the exit code.	./capstone-azure.sh run ci 'docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy:latest image --severity CRITICAL --exit-code 1 nginx:1.14 2>&1 | tail -n 12; echo exit=$?'	./capstone-aws.sh run ci 'docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy:latest image --severity CRITICAL --exit-code 1 nginx:1.14 2>&1 | tail -n 12; echo exit=$?'
3	Hunt for secrets in your Git history. Names such as TOKEN in code are false positives; you are looking for VALUES.	cd ~/capstone-app-$STUDENT_ID && git log -p | grep -nE "BEGIN .*PRIVATE KEY|AKIA[0-9A-Z]{16}|password *= *.[A-Za-z0-9]{6,}" | head	
4	List FIVE shortcuts in YOUR OWN lab that you would not accept in production, with the fix. Candidates: Jenkins over HTTP; jenkins user in the	Fill a table: shortcut / risk (attacker action) / Azure or AWS fix (for example Key Vault, private endpoint, managed identity, ACR with RBAC, WAF, environment approvals).	

#	What you do	Azure (Cloud Shell and portal)	AWS (CloudShell and console)
	docker group (root-equivalent); registry without TLS; Kubernetes Secret only base64; Run Command executes as root; public IPs on servers.		
5	Compare tools: complete the table in your notes: Bash script, Ansible, Bicep or ARM, Terraform, CloudFormation. Columns: imperative or declarative, idempotent, what it manages.	Notes	

Evidence to keep (screenshots, file names M#_what.png)
M8_scan.png (both scans), M8_shortcuts_table (your 5 rows).

Check question (answer in your own words, using your own output)
Is the provisioning script itself idempotent? What happens if you run network twice, and which tools would make that safe? (Expected: no, the second run fails or duplicates; declarative IaC such as Bicep, Terraform, CloudFormation reconciles to the desired state.)

M9 Monitoring, alerts and the audit trail

Course link	Days 5 and 6: monitoring and logging, observability
Time	30 min (15:30 to 16:00)
Exam skill	AZ-400: instrumentation strategy; AZ-104: monitor and maintain

The alert e-mail takes 5 to 10 minutes. Start the load now and continue with M10 while you wait.

#	What you do	Azure (Cloud Shell and portal)	AWS (CloudShell and console)
1	Find the alert definition.	Monitor > Alerts > Alert rules > alert-cpu-cap-<id>; Action groups > ag-cap-<id>.	CloudWatch > Alarms > cap-<id>-cpu-k8s. Open your mailbox and click Confirm subscription (SNS), otherwise no mail arrives.
2	Look at the normal CPU of the k8s server.	VM > Monitoring > Metrics > Percentage CPU.	EC2 > instance > Monitoring tab (or CloudWatch > Metrics).
3	Create load for 6 minutes.	./capstone-azure.sh load	./capstone-aws.sh load

#	What you do	Azure (Cloud Shell and portal)	AWS (CloudShell and console)
4	Watch the metric climb above 70 percent and the alert fire. Screenshot the fired state and the e-mail.	Monitor > Alerts (fired); mailbox.	CloudWatch > Alarms (In alarm); mailbox. Basic monitoring has 5 minute points, so allow 10 minutes.
5	Read the audit trail: who created the servers and when.	Monitor > Activity log, filter resource group, last 2 hours.	CloudTrail > Event history, event name RunInstances (can take 15 minutes to appear).
6	Read service logs.	./capstone-azure.sh run ci 'journalctl -u jenkins -n 8 --no-pager' ./capstone-azure.sh kube logs deploy/capstone-app --tail=5	./capstone-aws.sh run ci 'journalctl -u jenkins -n 8 --no-pager' ./capstone-aws.sh kube logs deploy/capstone-app --tail=5

Evidence to keep (screenshots, file names M#_what.png)
M9_alert_rule.png, M9_cpu_chart.png, M9_alert_fired.png, M9_mail.png, M9_audit.png.

Check question (answer in your own words, using your own output)
Write the alert you would set for the production version of this app: metric or log message, threshold, window, who is notified, and what they do first. What would have shown your most common lab error earlier? (Use a real error you hit today and its exact text.)

M10 Prove it, deliver it, clean it up

Course link	Days 4 and 6: change management, accountability, career readiness; all days
Time	50 min (16:00 to 16:50)
Exam skill	AZ-400: processes and communications; change management

This mission closes the day. Do the cleanup even if verify is not perfect: unfinished work can be rebuilt, a forgotten server cannot be un-billed.

#	What you do	Azure (Cloud Shell and portal)	AWS (CloudShell and console)
1	Run the automatic check. Screenshot the whole result (score, token, time).	./capstone-azure.sh verify	./capstone-aws.sh verify
2	Value stream, part 2: print lead time, wait time and flow efficiency. Compare with your manual estimate from M1.	./capstone-azure.sh report	./capstone-aws.sh report

#	What you do	Azure (Cloud Shell and portal)	AWS (CloudShell and console)
3	Write the deliverables in Section 8 (stakeholder message, change-request decision, case study, freelance offer).	Word or Markdown, saved in your evidence folder	
4	Live oral follow-up: the trainer picks one question and asks you to re-run one command on screen.	Trainer	
5	CLEAN UP. Type the confirmation when asked. Wait for the word CLEAN.	./cleanup-azure.sh	./cleanup-aws.sh
6	Check billing and remove tokens.	Cost Management > Cost analysis for the day. Delete the GitHub token. Optional: delete the resource group cloud-shell-storage-* at course end.	Billing > Free Tier usage; EC2 console shows no instances; SNS e-mail can be ignored. Delete the GitHub token.

Evidence to keep (screenshots, file names M#_what.png)

M10_verify.png (score + token + time), M10_report.png, M10_clean.png (the CLEAN line), billing screenshot.

Check question (answer in your own words, using your own output)

It is Friday 4:40 pm. A manager asks you to release version 2.0 today. Your pipeline is green, one test is skipped and nobody reviewed the pull request. State your decision in one sentence with three reasons, and name the controls that would let you say yes safely. (Section 8 holds the full task.)

8. Professional deliverables (ethics, stakeholder, freelancing)

These cover the soft-skill and freelancing parts of the course PDF. Each is short, uses YOUR evidence, and is written in your own words. Keep them in the evidence folder; they are marked (10 marks in total).

D1. Stakeholder update (Day 1 soft skill, 120 words maximum)

Scenario: your M5 red build delayed a release by two hours. Write the message to a non-technical manager. Use this order: headline (one sentence), business impact, cause in plain words, what you did, new time and who owns the next step. Quote your real build number and the time. No jargon without a one-line explanation.

D2. Change-request decision (Day 4 soft skill; the Friday 4:40 pm scenario)

A manager asks for version 2.0 in production today. The pipeline is green, one automated test is skipped, nobody reviewed the pull request. Write: (1) your decision in one sentence, (2) three reasons, (3) a reply message of at most 120 words in plain business language, (4) the controls that would let you say yes safely, and which of them you actually have in this lab. Use a real build number or commit hash from your own repository as the reference.

Control	Azure name	AWS name	Do you have it in the lab?
Required reviewer on the pull request	Azure Repos branch policy	GitHub branch protection / CodeCommit approval rules	GitHub branch protection can be enabled (yes/no)
Block release when tests fail or are skipped	Build validation, quality gate	Pipeline stage condition	Jenkins: red Test stage (yes)
Human approval before production	Environment approval and checks	Manual approval action	Jenkins Approval stage (yes)
Safe rollout and quick rollback	Deployment slot, traffic split, rollout undo	Blue/green, weighted routing, rollout undo	Kubernetes rollout undo (yes)
Access review	Entra access reviews, PIM	IAM Access Analyzer	No (write what you would do)

D3. Portfolio case study and README (Days 1, 2, 3, 5, 6 freelancing and career)

- README of your capstone-app repository: what it is (a training demo, not client work), the architecture picture, how to run it, what the pipeline does, what you learned, what you would improve. Pin the repository on your GitHub profile.
- One-page case study in STAR form (situation, task, action, result) with numbers from your own run: lead time, flow efficiency, build number, time to recover from the 9.9 release, alert time.
- Three lines for your freelance profile that are TRUE and provable from this lab (for example: "Built a Jenkins pipeline with approval gate and image scan; deployed to Kubernetes with zero-downtime rollout and rollback on Azure and AWS free tier").

D4. Freelance offer and scope (Days 2, 3 and 5 freelancing)

Write a one-page offer for a small client who needs "a containerized app with a basic CI/CD pipeline". Fill the table with YOUR hours and YOUR hourly rate (do not copy anyone's number). Define scope, what is excluded, how many revisions, acceptance test (your verify-style check), and the rule for out-of-scope requests.

Package	You deliver	Your estimate (hours)	Out of scope
Starter	Dockerfile, repo, README, local run		Cloud hosting, monitoring
Standard	Starter + Jenkins pipeline with tests and image build		Kubernetes, scanning
Pro	Standard + scan, approval gate, Kubernetes deployment, alert		24/7 support, client cloud costs

D5. Azure versus AWS comparison (pairs in the track-split option)

Question	Azure result	AWS result
Lead time of all (from report)		
Flow efficiency		
Which step was hardest to understand?		
Which console was easier for the alert?		
Cost shown for the day		

9. Assessment, trainer verification and answer key

Marking scheme (100 marks)

Part	Marks	How it is marked
M0 and M1: setup, Git, pull request, manual estimate	10	Evidence shows the student's id, repo URL and merged PR.
M2, M3, M4: network, identity, compute, Ansible	15	Rules, denied write, changed=0, drift repaired.
M5 and M6: Jenkins and containers	20	Green run with approval, red run with failing line, layer cache answer.
M7: Kubernetes	15	Zero-downtime loop, 9.9 events, rollback, heal, answer.
M8 and M9: security, IaC, monitoring	15	Five shortcuts table, alert fired, audit trail.
verify score	10	Score divided by total, times 10. Token and time must match the trainer sheet.
Deliverables D1 to D4	10	Own words, own evidence, correct Azure or AWS names.
Cleanup proof	5	The CLEAN line and a billing screenshot.

Per-question scoring (same rule as the Day 4 and 5 descriptive questions)

Criterion	Marks of 10
Personal evidence is present, specific and consistent (id, token, IP, build number, real output)	3
Technical explanation is correct and uses course terms precisely	3
Azure or AWS mapping is correct and relevant	2
Reasoning, trade-offs and honest self-assessment	2

How copying is made visible

- Every app page shows owner=<id> | token=<6 characters>. The token is generated from the id and the start time on the student's own shell, and is printed again by verify with the time. Screenshots without the right token get no credit for that mission.
- Ask each student to **share their screen** and re-run one command live (for example kube get pods or run ci "docker history ...").
- Use the oral follow-up bank: "Which command did you run just before that output?"; "If I delete that pod now, what will you see and why? Show me."; "Which line of that file would you change first to make this safer?"; "In Azure or AWS, where would I click to see the same thing?"; "What would you do differently for 500 users instead of 5?"; "Tell me about a mistake you made on this step."
- **Honest limit:** no written task is safe from AI help, because a student can paste their own output into a tool. This design raises the effort and makes copying visible; the live screen share and oral follow-up are the real protection.

Answer key for the check questions

Mission	Expected answer (short)
M0	No. A budget only alerts. Only deleting or deallocating resources stops cost; some items (disks, public IPs) cost even when a VM is stopped.
M1	A PR with review is a gate before code reaches main. Azure Repos: branch policy (required reviewers, build validation, linked work items); GitHub: branch protection.
M2	Least privilege and attack surface: nothing but your IP reaches the app; the registry is plain HTTP and so only the subnet may use it; SSH is replaced by Bastion, just-in-time access or Run Command (AWS Session Manager or Run Command).
M3	Secrets User is read-only and scoped to one vault, so a stolen server identity cannot write or touch other resources. A lock blocks deletion even for an Owner; RBAC controls who may act, a lock guards against mistakes by people who are allowed.
M4	Running the playbook again changes nothing when the machine already matches the desired state (idempotence), and it repairs drift. Declarative: Bicep, ARM, Terraform, CloudFormation.
M5	Stages after Test are skipped because the default condition is "previous stage succeeded". No new tag is pushed. Azure Pipelines: stages with dependsOn, condition: succeeded(), branch policy with build validation, environment approval before deploy.
M6	Only the COPY app layer and later layers rebuild; base image and dependency layers use the cache because dependencies are installed before the code is copied. Copying code first would invalidate the install layer on every change. Slower builds mean more paid agent minutes; ACR Tasks or layer caching reduce cost.
M7	maxUnavailable 0 plus maxSurge 1 keeps the old pods until a new pod passes its readiness probe; the 9.9 pod never becomes Ready, so traffic stays on old pods. Azure DevOps: environment approvals and checks (and monitoring gates), then rollback with kubectl rollout undo or redeploy the previous image.
M8	No: the script is imperative and fails or duplicates on a second run. Declarative IaC reconciles to a desired state. Kubernetes Secrets are base64, not encryption: use encryption at rest, RBAC, and Key Vault or Secrets Manager integration.
M9	A good answer names a metric or log message, a threshold, a window, a recipient and a first action, for example CPU above 70 percent for 5 minutes, plus pod restart count or a log message such as ImagePullBackOff. Azure Monitor, Container insights, Application Insights or CloudWatch/Container Insights.
M10	Do not release now. Reasons: unreviewed code, a skipped test means unknown quality, Friday late release leaves no recovery time. Safe yes needs branch policy review, environment approval, a canary or slot with traffic split and a tested rollback plan. In the lab the student has Jenkins approval, a red Test stage and rollout undo; no access review.

Optional stretch ideas (fast students)

Add a Jenkins webhook (Day 2 workbook) so a push triggers a build. Add a second Ansible play that installs nginx. Enable GitHub branch protection and show the blocked merge. Replace the NodePort with an Ingress after re-enabling Traefik (needs a larger size). Write the same environment as a Bicep or CloudFormation file on paper.

10. Troubleshooting

Symptom	Likely cause	What to do
check says the size is unavailable (Azure)	Region does not offer the free size to your subscription.	Try export LOCATION= another region or export VM_SIZE=Standard_B2ats_v2, then check.
check says the type is not free-tier eligible (AWS)	Account type uses different eligible instances.	Use one of the types the message lists (export INSTANCE_TYPE=). Only use CONFIRM_PAID=yes if the trainer agrees.
Not signed in	Shell session expired.	Close and reopen the shell, re-export variables.
wait times out	Bootstrap still running or a package step failed.	run ci 'tail -n 40 /var/log/capstone-bootstrap.log'. Slow Jenkins install is normal; failed apt or curl usually means a transient network error: wait and run wait again.
Jenkins or the app does not open in the browser	Your public IP changed (VPN, mobile data) or the rule uses another address.	Open the rule in the console and edit the source to your current IP /32.
Azure "run command conflict"	Another Run Command is still running.	Wait 30 seconds and retry.
Azure secret set fails in identity	Role assignment has not propagated.	The script retries for 3 minutes. If it still fails, run identity again is NOT safe; set the secret in the portal, or use cleanup and restart.
AWS commands say the instance is not found or SSM not online	Agent not registered yet or profile not attached.	Wait 3 minutes; check EC2 > instance > IAM role.
K3s node NotReady or pods Pending	Out of memory on a 1 GB server.	kube get nodes; run k8s 'free -h; journalctl -u k3s -n 30 --no-pager'. Keep replicas at 2 or less.
Pod ImagePullBackOff on a valid tag	k8s server cannot reach the registry.	run k8s 'curl -s http://10.50.1.10:5000/v2/_catalog'; check the 5000 rule from the subnet.
Jenkins very slow or build killed	Memory pressure on the CI server.	Wait; run one build at a time. If needed use a larger free-eligible size or ask the trainer.
Alert e-mail never arrives	AWS: subscription not confirmed. Azure: mail in junk, wrong EMAIL.	Confirm the SNS subscription; check the action group address; allow 10 minutes.
Cleanup says something is left	Dependency order (security group before instances end, locks).	Run the cleanup script again; then look at the Resource groups or EC2/VPC lists filtered by tag.

Appendix A: capstone-azure.sh (Azure Cloud Shell)

Upload to Azure Cloud Shell (Bash). Also delivered as a separate file.

```
#!/usr/bin/env bash
#
================================================================================
=====
# capstone-azure.sh  -  DevOps Bootcamp Capstone (AZ-400 + AZ-104),
Azure edition
# Run in Azure Cloud Shell (Bash). Free-account sizes. One script, many
phases.
# Cleanup: cleanup-azure.sh   (always run it before you leave)
#
================================================================================
=====
# ============================== common helpers
==============================
set -uo pipefail
c_g=$'\033[1;32m'; c_y=$'\033[1;33m'; c_r=$'\033[1;31m';
c_p=$'\033[1;35m'; c_0=$'\033[0m'
say()  { echo "${c_p}==>${c_0} $*"; }
ok()   { echo "${c_g}[ OK ]${c_0} $*"; }
warn() { echo "${c_y}[WARN]${c_0} $*"; }
die()  { echo "${c_r}[FAIL]${c_0} $*" >&2; exit 1; }
need() { command -v "$1" >/dev/null 2>&1 || die "Missing tool: $1"; }

STUDENT_ID="${STUDENT_ID:-}"      # 3-8 lowercase letters/digits, e.g.
amk07
MY_IP="${MY_IP:-}"                # your PUBLIC IP as seen by your
browser (https://ifconfig.me)
EMAIL="${EMAIL:-}"                # where the CPU alert e-mail is sent
REG_IP="10.50.1.10"               # private IP of the CI server (Jenkins
+ registry)
K8S_IP="10.50.1.11"               # private IP of the Kubernetes (K3s)
server

save() { # save KEY VALUE  -> persists to the state file
  { grep -v "^$1=" "$STATE" 2>/dev/null || true; echo "$1=\"$2\""; } >
"$STATE.tmp" && mv "$STATE.tmp" "$STATE"
}

init_workspace() {
  [[ "$STUDENT_ID" =~ ^[a-z0-9]{3,8}$ ]] || die "Set STUDENT_ID first
(3-8 lowercase letters/digits), e.g.: export STUDENT_ID=amk07"
  P="cap-$STUDENT_ID"
  WORK="$HOME/capstone-$CLOUD-$STUDENT_ID"; mkdir -p "$WORK"
  STATE="$WORK/state.env"; TIMELINE="$WORK/timeline.log"; touch "$STATE"
"$TIMELINE"
  # shellcheck disable=SC1090
  . "$STATE"
  if [ -z "${TOKEN:-}" ]; then
```

```
    TOKEN=$(printf '%s' "$STUDENT_ID-$(date +%s)" | sha256sum | cut -c1-
6); save TOKEN "$TOKEN"
  fi
}

# Every phase is timed: this feeds the value-stream report (Day 1
skill).
phase_run() { # phase_run NAME FUNCTION
  local n=$1 f=$2 s e rc; s=$(date +%s); say "PHASE $n starting"
  "$f"; rc=$?; e=$(date +%s); echo "$n $s $e $rc" >> "$TIMELINE"
  [ $rc -eq 0 ] && ok "PHASE $n finished in $((e-s))s" || warn "PHASE $n
ended with code $rc"
  return $rc
}

phase_report() {
  say "Value-stream report for $STUDENT_ID (Day 1: lead time, process
time, wait time)"
  awk '
    { n[NR]=$1; s[NR]=$2; e[NR]=$3; pt+=($3-$2); if(NR==1||$2<first)
first=$2; if($3>last) last=$3 }
    END {
      if (NR==0) { print "no phases recorded yet"; exit }
      printf "%-12s %8s\n", "phase", "minutes";
      for (i=1;i<=NR;i++) printf "%-12s %8.1f\n", n[i], (e[i]-s[i])/60;
      lt=last-first; wt=lt-pt;
      printf "\nProcess time (script working) : %6.1f min\n", pt/60;
      printf "Wait time (you + the cloud)   : %6.1f min\n", wt/60;
      printf "Lead time (first to last)     : %6.1f min\n", lt/60;
      if (lt>0) printf "Flow efficiency               : %6.1f %%\n",
100*pt/lt;
    }' "$TIMELINE"
  echo "Your token: $TOKEN   Student: $STUDENT_ID   Time: $(date
'+%F %T %Z')"
}

# -------------------------- the app bundle --------------------------
------
# Everything the servers need (app, tests, Dockerfile, Jenkinsfile, k8s
YAML,
# Ansible playbook, bootstrap) is generated here and shipped inside
user-data.
make_bundle() {
  local B="$WORK/bundle"; rm -rf "$B"; mkdir -p "$B/app" "$B/tests"
"$B/k8s" "$B/ansible"
  : > "$B/app/__init__.py"
  cat > "$B/app/main.py" <<'PY'
import os
import socket
from flask import Flask

app = Flask(__name__)
VERSION = os.environ.get("APP_VERSION", "dev")
```

```
OWNER = os.environ.get("OWNER", "student")
TOKEN = os.environ.get("TOKEN", "none")


@app.route("/")
def home():
    return f"Capstone app v{VERSION} | owner={OWNER} | token={TOKEN}\n"


@app.route("/health")
def health():
    return "ok"


@app.route("/host")
def host():
    return socket.gethostname() + "\n"
PY
  cat > "$B/tests/test_app.py" <<'PY'
from app.main import app


def test_home():
    assert b"Capstone app" in app.test_client().get("/").data


def test_health():
    assert app.test_client().get("/health").data == b"ok"
PY
  printf 'flask==3.0.3\ngunicorn==22.0.0\npytest==8.3.2\n' >
"$B/requirements.txt"
  printf 'flask==3.0.3\ngunicorn==22.0.0\n' > "$B/requirements-
runtime.txt"
  printf
'.git\n.venv\n__pycache__\n.pytest_cache\ntests\nk8s\nansible\nbootstrap
.sh\nJenkinsfile\n' > "$B/.dockerignore"
  cat > "$B/Dockerfile" <<'DF'
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements-runtime.txt .
RUN pip install --no-cache-dir -r requirements-runtime.txt
COPY app ./app
ARG APP_VERSION=1.0
ENV APP_VERSION=$APP_VERSION
RUN useradd --create-home --uid 10001 appuser
USER appuser
EXPOSE 5000
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "app.main:app"]
DF
  cat > "$B/Jenkinsfile" <<'JF'
pipeline {
  agent any
```

```
  options { timestamps(); timeout(time: 15, unit: 'MINUTES') }
  environment { REGISTRY = 'localhost:5000'; IMAGE = 'capstone-app' }
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Test') {
      steps {
        sh '''
          python3 -m venv .venv
          .venv/bin/pip install -q -r requirements.txt
          mkdir -p reports
          .venv/bin/python -m pytest -q --junitxml=reports/junit.xml
        '''
      }
      post { always { junit 'reports/junit.xml' } }
    }
    stage('Build image') {
      steps { sh 'docker build -t $REGISTRY/$IMAGE:$BUILD_NUMBER --
build-arg APP_VERSION=$BUILD_NUMBER .' }
    }
    stage('Scan image') {
      // --exit-code 0 only reports. Change to 1 to FAIL the build on
CRITICAL findings.
      steps { sh 'docker run --rm -v
/var/run/docker.sock:/var/run/docker.sock aquasec/trivy:latest image --
severity CRITICAL --exit-code 0 $REGISTRY/$IMAGE:$BUILD_NUMBER' }
    }
    stage('Approval') {
      steps { timeout(time: 10, unit: 'MINUTES') { input message:
'Publish this build to the registry?' } }
    }
    stage('Publish') { steps { sh 'docker push
$REGISTRY/$IMAGE:$BUILD_NUMBER' } }
  }
  post { failure { echo 'capstone-app: build FAILED - open Console
Output and Test Result' } }
}
JF
  cat > "$B/k8s/deployment.yaml" <<'KY'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: capstone-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: capstone-app
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
```

```
    metadata:
      labels:
        app: capstone-app
    spec:
      containers:
        - name: app
          image: REGISTRY_IP:5000/capstone-app:1.0
          ports:
            - containerPort: 5000
          env:
            - name: OWNER
              value: "OWNER_ID"
            - name: TOKEN
              value: "TOKEN_VALUE"
          readinessProbe:
            httpGet: { path: /health, port: 5000 }
            initialDelaySeconds: 3
            periodSeconds: 5
          livenessProbe:
            httpGet: { path: /health, port: 5000 }
            initialDelaySeconds: 10
            periodSeconds: 10
          resources:
            requests: { cpu: 50m, memory: 48Mi }
            limits: { cpu: 250m, memory: 128Mi }
---
apiVersion: v1
kind: Service
metadata:
  name: capstone-app
spec:
  type: NodePort
  selector:
    app: capstone-app
  ports:
    - port: 80
      targetPort: 5000
      nodePort: 30080
KY
  cat > "$B/ansible/baseline.yml" <<'AY'
---
- name: Capstone baseline (must be idempotent)
  hosts: localhost
  connection: local
  become: true
  vars:
    swap_mb: 2048
    install_docker: false
  tasks:
    - name: Refresh apt cache (not counted as a change)
      ansible.builtin.apt:
        update_cache: true
        cache_valid_time: 3600
```

```
      changed_when: false
    - name: Base tools
      ansible.builtin.apt:
        name: [git, curl, jq, unzip]
        state: present
    - name: CI tools (Docker and Python venv)
      ansible.builtin.apt:
        name: [docker.io, python3-venv]
        state: present
      when: install_docker | bool
    - name: Docker service enabled and started
      ansible.builtin.service:
        name: docker
        state: started
        enabled: true
      when: install_docker | bool
    - name: Create swap file
      ansible.builtin.command: "fallocate -l {{ swap_mb }}M /swapfile"
      args:
        creates: /swapfile
    - name: Swap file permissions
      ansible.builtin.file:
        path: /swapfile
        owner: root
        group: root
        mode: "0600"
    - name: Format swap file
      ansible.builtin.shell: "blkid -t TYPE=swap /swapfile >/dev/null
2>&1 || (mkswap /swapfile && echo formatted)"
      register: fmt
      changed_when: "'formatted' in fmt.stdout"
    - name: Enable swap
      ansible.builtin.shell: "swapon --show=NAME --noheadings | grep -qx
/swapfile || (swapon /swapfile && echo enabled)"
      register: son
      changed_when: "'enabled' in son.stdout"
    - name: Persist swap in fstab
      ansible.builtin.lineinfile:
        path: /etc/fstab
        line: "/swapfile none swap sw 0 0"
    - name: Marker file proving Ansible managed this server
      ansible.builtin.copy:
        dest: /etc/capstone-baseline.txt
        content: "managed-by=ansible\nswap_mb={{ swap_mb }}\n"
        mode: "0644"
AY
  cat > "$B/bootstrap.sh" <<'BOOT_EOF'
#!/bin/bash
# usage: bootstrap.sh ROLE REG_IP CLOUD OWNER TOKEN     (runs once, as
root, from user-data)
set -euo pipefail
ROLE="$1"; REG_IP="$2"; CLOUD="$3"; OWNER="$4"; TOKEN="$5"
exec >> /var/log/capstone-bootstrap.log 2>&1
```

```
set -x
export DEBIAN_FRONTEND=noninteractive
echo "$ROLE" > /etc/capstone-role
apt-get update -y
apt-get install -y ansible-core
DOCKER=false
if [ "$ROLE" = "ci" ]; then DOCKER=true; fi
ansible-playbook -c local -i localhost, -e install_docker=$DOCKER
/opt/capstone/ansible/baseline.yml
if [ "$CLOUD" = "aws" ]; then apt-get install -y awscli || snap install
aws-cli --classic; fi

if [ "$ROLE" = "ci" ]; then
  cd /opt/capstone
  docker run -d --name registry --restart=always -p 5000:5000 -v
registrydata:/var/lib/registry registry:2
  sleep 5
  for v in 1.0 2.0; do
    docker build -t "localhost:5000/capstone-app:$v" --build-arg
"APP_VERSION=$v" .
    docker push "localhost:5000/capstone-app:$v"
  done
  apt-get install -y openjdk-21-jre fontconfig
  install -d -m 0755 /etc/apt/keyrings
  curl -fsSL -o /etc/apt/keyrings/jenkins-keyring.asc
https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
  echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]
https://pkg.jenkins.io/debian-stable binary/" >
/etc/apt/sources.list.d/jenkins.list
  apt-get update -y
  apt-get install -y jenkins
  usermod -aG docker jenkins
  mkdir -p /etc/systemd/system/jenkins.service.d
  printf '[Service]\nEnvironment="JAVA_OPTS=-Djava.awt.headless=true -
Xmx512m"\n' > /etc/systemd/system/jenkins.service.d/override.conf
  systemctl daemon-reload
  systemctl enable jenkins
  systemctl restart jenkins
fi

if [ "$ROLE" = "k8s" ]; then
  mkdir -p /etc/rancher/k3s
  cat > /etc/rancher/k3s/registries.yaml <<REG_EOF
mirrors:
  "$REG_IP:5000":
    endpoint:
      - "http://$REG_IP:5000"
REG_EOF
  sed -i "s/REGISTRY_IP/$REG_IP/; s/OWNER_ID/$OWNER/;
s/TOKEN_VALUE/$TOKEN/" /opt/capstone/k8s/deployment.yaml
  curl -sfL https://get.k3s.io | INSTALL_K3S_EXEC="server --disable
traefik" sh -
```

```
  for i in $(seq 1 60); do k3s kubectl get nodes 2>/dev/null | grep -q '
Ready' && break; sleep 5; done
fi
touch /var/lib/capstone-ready
BOOT_EOF
  chmod +x "$B/bootstrap.sh"
  tar czf "$WORK/bundle.tgz" -C "$B" .
  base64 -w 76 "$WORK/bundle.tgz" > "$WORK/bundle.b64"
}

make_userdata() { # make_userdata ROLE OUTFILE   (user-data = a shell
script that unpacks the bundle)
  local role=$1 out=$2
  {
    echo '#!/bin/bash'
    echo 'mkdir -p /opt/capstone'
    echo "cat > /tmp/bundle.b64 <<'B64'"
    cat "$WORK/bundle.b64"
    echo 'B64'
    echo 'base64 -d /tmp/bundle.b64 | tar xz -C /opt/capstone'
    echo "bash /opt/capstone/bootstrap.sh $role $REG_IP $CLOUD
$STUDENT_ID $TOKEN"
  } > "$out"
  local size; size=$(stat -c %s "$out")
  [ "$size" -lt "$USERDATA_MAX" ] || die "user-data is $size bytes
(limit $USERDATA_MAX)"
  ok "user-data for $role: $size bytes"
}


phase_export() { # writes a Git-ready copy of the app (Mission 1: you
push it to YOUR GitHub)
  make_bundle
  local dst="$HOME/capstone-app-$STUDENT_ID"; mkdir -p "$dst"; cp -a
"$WORK/bundle/." "$dst/"
  printf '.venv/\n__pycache__/\n.pytest_cache/\n*.pyc\n' >
"$dst/.gitignore"
  printf '# capstone-app (%s)\n\nDemo project from the DevOps bootcamp
capstone. Not client work.\n' "$STUDENT_ID" > "$dst/README.md"
  ok "Git-ready folder: $dst"
}

need_my_ip() {
  [[ "$MY_IP" =~ ^([0-9]{1,3}\.){3}[0-9]{1,3}$ ]] || die "Set MY_IP to
your public IP (open https://ifconfig.me in YOUR browser), e.g.: export
MY_IP=203.0.113.7"
  [ "$MY_IP" != "0.0.0.0" ] || die "MY_IP must be your own address,
never 0.0.0.0"
}

# ============================== Azure specifics
==============================
CLOUD=azure
```

```
LOCATION="${LOCATION:-southeastasia}"      # choose a region where your
free-account VM size is offered
VM_SIZE="${VM_SIZE:-Standard_B1s}"         # free-account size (750
h/month for 12 months)
USERDATA_MAX=60000                         # custom-data limit is 64 KB

usage() {
cat <<USAGE
Capstone (Azure)  -  usage:  ./capstone-azure.sh <phase> [args]

  Set these first:  export STUDENT_ID=amk07  MY_IP=<your public IP>
EMAIL=<you@mail>
  Optional:         export LOCATION=southeastasia  VM_SIZE=Standard_B1s

  all        check, network, compute, wait, identity, monitor, status
(about 15 min)
  check      login, region, VM size and input checks (nothing is
created)
  network    resource group, VNet, subnet, NSG with least-privilege
rules
  compute    two Ubuntu 24.04 VMs (ci = Jenkins + registry, k8s = K3s),
Ansible baseline
  wait       wait until both servers finish their bootstrap
  identity   Key Vault (RBAC), secret, managed-identity role, resource
lock, policy
  monitor    action group + CPU alert rule
  status     URLs and where to click in the portal
  run   ci|k8s "<command>"     run a shell command on a server (no SSH
needed)
  kube  <kubectl arguments>    run kubectl on the k8s server
  ansible ci|k8s               re-run the Ansible baseline and show the
recap
  load                         6 minutes of CPU load on the k8s server
(alert test)
  verify     automatic check of the whole capstone (prints your score
and token)
  export     write a Git-ready copy of the app to ~/capstone-app-<id>
(Mission 1)
  secret [read|write]          read the secret with the server identity,
or try to write it (must fail)
  report     value-stream report (lead time, flow efficiency)
USAGE
}

az_vmname() { case "$1" in ci) echo "$P-ci";; k8s) echo "$P-k8s";; *)
die "role must be ci or k8s";; esac; }

remote() { # remote ROLE "command"  (Azure Run Command: no open SSH port
required)
  az vm run-command invoke -g "$RG" -n "$(az_vmname "$1")" --command-id
RunShellScript \
    --scripts "export KUBECONFIG=/etc/rancher/k3s/k3s.yaml; $2" --query
'value[0].message' -o tsv
```

```
}

set_names() {
  RG="rg-$P"; VNET="vnet-$P"; SUBNET="snet-app"; NSG="nsg-$P"
  if [ -z "${SUFFIX:-}" ]; then SUFFIX=$(printf '%s' "$TOKEN" | cut -c1-
5); save SUFFIX "$SUFFIX"; fi
  KV="kv${STUDENT_ID}${SUFFIX}"
}

phase_check() {
  need az; need jq; need curl
  az account show >/dev/null 2>&1 || die "Not signed in. Open Azure
Cloud Shell (Bash) from the portal; it signs you in."
  need_my_ip
  local sku; sku=$(az vm list-skus --location "$LOCATION" --size
"$VM_SIZE" --resource-type virtualMachines \
        --query '[?length(restrictions)==`0`].name | [0]' -o tsv
2>/dev/null)
  [ -n "$sku" ] || die "$VM_SIZE is not available to you in $LOCATION.
Try another LOCATION or VM_SIZE (B2ats_v2, B2pts_v2)."
  ok "Subscription: $(az account show --query name -o tsv)  Region:
$LOCATION  Size: $VM_SIZE"
  warn "Free account: check Cost Management + Billing > Free services.
Public IPs and disks can cost cents per hour."
  for ns in Microsoft.Compute Microsoft.Network Microsoft.KeyVault
Microsoft.Insights Microsoft.PolicyInsights; do
    az provider register --namespace "$ns" >/dev/null 2>&1
  done
  ok "Resource providers registering in the background"
}

phase_network() {
  need_my_ip
  az group create -n "$RG" -l "$LOCATION" --tags project=capstone
owner="$STUDENT_ID" -o none && ok "Resource group $RG"
  az network vnet create -g "$RG" -n "$VNET" --address-prefix
10.50.0.0/16 \
    --subnet-name "$SUBNET" --subnet-prefix 10.50.1.0/24 --tags
project=capstone -o none && ok "VNet $VNET"
  az network nsg create -g "$RG" -n "$NSG" --tags project=capstone -o
none
  az network nsg rule create -g "$RG" --nsg-name "$NSG" -n allow-
jenkins-from-me --priority 100 --direction Inbound \
    --access Allow --protocol Tcp --source-address-prefixes "$MY_IP/32"
--destination-port-ranges 8080 -o none
  az network nsg rule create -g "$RG" --nsg-name "$NSG" -n allow-app-
from-me --priority 110 --direction Inbound \
    --access Allow --protocol Tcp --source-address-prefixes "$MY_IP/32"
--destination-port-ranges 30080 -o none
  az network nsg rule create -g "$RG" --nsg-name "$NSG" -n allow-
registry-from-subnet --priority 120 --direction Inbound \
    --access Allow --protocol Tcp --source-address-prefixes
10.50.1.0/24 --destination-port-ranges 5000 -o none
```

```
  az network vnet subnet update -g "$RG" --vnet-name "$VNET" -n
"$SUBNET" --network-security-group "$NSG" -o none
  ok "NSG $NSG attached to the subnet. There is NO SSH rule: servers are
managed with Run Command."
}

phase_compute() {
  make_bundle
  make_userdata ci "$WORK/ud-ci.sh"; make_userdata k8s "$WORK/ud-k8s.sh"
  local role ip
  for role in ci k8s; do
    ip=$REG_IP; [ "$role" = k8s ] && ip=$K8S_IP
    az vm create -g "$RG" -n "$(az_vmname $role)" --image
Canonical:ubuntu-24_04-lts:server:latest --size "$VM_SIZE" \
      --admin-username azureuser --generate-ssh-keys --vnet-name "$VNET"
--subnet "$SUBNET" \
      --private-ip-address "$ip" --public-ip-sku Standard --public-ip-
address "$(az_vmname $role)-pip" --nsg "" \
      --assign-identity '[system]' --custom-data "$WORK/ud-$role.sh" \
      --storage-sku Premium_LRS --os-disk-size-gb 64 --tags
project=capstone owner="$STUDENT_ID" role="$role" \
      --no-wait -o none || die "az vm create failed for $role"
  done
  for role in ci k8s; do az vm wait -g "$RG" -n "$(az_vmname $role)" --
created; done
  ok "Both VMs exist. They now install software on their own (about 8 to
12 minutes). Use: ./capstone-azure.sh wait"
}

phase_wait() {
  local role n=0 out
  for role in ci k8s; do
    n=0
    while [ $n -lt 30 ]; do
      out=$(remote $role 'test -f /var/lib/capstone-ready && echo READY
|| echo WAIT' 2>/dev/null | tr -d '\r' | grep -Eo 'READY|WAIT' | head -
n1)
      [ "$out" = READY ] && { ok "$role server is ready"; break; }
      n=$((n+1)); say "$role still installing ($n/30) ..."; sleep 20
    done
    [ "$out" = READY ] || die "$role not ready. Read the
log: ./capstone-azure.sh run $role 'tail -n 40 /var/log/capstone-
bootstrap.log'"
  done
}

phase_identity() {
  local me kvid vmpid t
  az keyvault create -g "$RG" -n "$KV" -l "$LOCATION" --enable-rbac-
authorization true --retention-days 7 --tags project=capstone -o none \
     || die "Key Vault creation failed"
  kvid=$(az keyvault show -n "$KV" --query id -o tsv)
```

```
  me="${ME_OID:-$(az ad signed-in-user show --query id -o tsv
2>/dev/null)}"
  [ -n "$me" ] || die "Cannot read your object id. Set it manually:
export ME_OID=<object id from Entra ID > Users>"
  az role assignment create --assignee-object-id "$me" --assignee-
principal-type User \
     --role "Key Vault Secrets Officer" --scope "$kvid" -o none && ok
"You: Key Vault Secrets Officer (scope = this vault only)"
  vmpid=$(az vm show -g "$RG" -n "$(az_vmname ci)" --query
identity.principalId -o tsv)
  az role assignment create --assignee-object-id "$vmpid" --assignee-
principal-type ServicePrincipal \
     --role "Key Vault Secrets User" --scope "$kvid" -o none && ok "CI
VM managed identity: Key Vault Secrets User (read only)"
  for t in $(seq 1 12); do
    az keyvault secret set --vault-name "$KV" --name db-password --value
"$(openssl rand -base64 12)" -o none 2>/dev/null && { ok "Secret db-
password stored"; break; }
    say "waiting for role propagation ($t/12) ..."; sleep 15
  done
  az lock create --name lock-kv --lock-type CanNotDelete -g "$RG" --
resource-name "$KV" \
     --resource-type Microsoft.KeyVault/vaults -o none && ok
"CanNotDelete lock on the vault"
  az policy assignment create --name allowed-loc --display-name "Allowed
locations (capstone)" \
     --policy e56962a6-4747-49cd-b67b-bf8b01975c4c --params
"{\"listOfAllowedLocations\":{\"value\":[\"$LOCATION\"]}}" \
     --scope "$(az group show -n "$RG" --query id -o tsv)" -o none
2>/dev/null \
     && ok "Policy: only $LOCATION allowed in this resource group" ||
warn "Policy assignment skipped (optional)"
}

phase_monitor() {
  [ -n "$EMAIL" ] || die "Set EMAIL first: export EMAIL=you@example.com"
  local vmid; vmid=$(az vm show -g "$RG" -n "$(az_vmname k8s)" --query
id -o tsv)
  az monitor action-group create -g "$RG" -n "ag-$P" --short-name
capstone --action email student "$EMAIL" -o none \
     && ok "Action group ag-$P -> $EMAIL"
  az monitor metrics alert create -g "$RG" -n "alert-cpu-$P" --scopes
"$vmid" --condition "avg Percentage CPU > 70" \
     --window-size 5m --evaluation-frequency 1m --action "ag-$P" --
description "Capstone: k8s server CPU above 70 percent" -o none \
     && ok "Alert rule alert-cpu-$P (CPU > 70 percent for 5 minutes)"
}

phase_status() {
  local ci k8
  ci=$(az vm list-ip-addresses -g "$RG" -n "$(az_vmname ci)" --query
'[0].virtualMachine.network.publicIpAddresses[0].ipAddress' -o tsv)
```

```
  k8=$(az vm list-ip-addresses -g "$RG" -n "$(az_vmname k8s)" --query
'[0].virtualMachine.network.publicIpAddresses[0].ipAddress' -o tsv)
  echo; echo "Jenkins (from YOUR browser/IP only) : http://$ci:8080"
  echo "App on Kubernetes (NodePort)        : http://$k8:30080"
  echo "Portal: resource group  rg-$P  |  Key Vault: $KV  |  Token:
$TOKEN"
  echo "Jenkins unlock password : ./capstone-azure.sh run ci 'cat
/var/lib/jenkins/secrets/initialAdminPassword'"
}

phase_ansible() { remote "$1" "ansible-playbook -c local -i localhost, -
e install_docker=$([ "$1" = ci ] && echo true || echo false)
/opt/capstone/ansible/baseline.yml" | tail -n 25; }

phase_secret() { # secret [read|write]  - uses the VM managed identity,
no password anywhere
  local tok="TOK=\$(curl -s -H Metadata:true
'http://169.254.169.254/metadata/identity/oauth2/token?api-version=2018-
02-01&resource=https%3A%2F%2Fvault.azure.net' | jq -r .access_token);"
  if [ "${1:-read}" = write ]; then
    say "Trying to WRITE a secret with the read-only identity (must be
refused)"
    remote ci "$tok curl -s -o /dev/null -w 'HTTP %{http_code}\n' -X PUT
-H \"Authorization: Bearer \$TOK\" -H 'Content-Type: application/json' -
d '{\"value\":\"x\"}' 'https://$KV.vault.azure.net/secrets/not-
allowed?api-version=7.4'"
  else
    say "Reading the secret with the VM managed identity (only the
length is shown)"
    remote ci "$tok echo secret length: \$(curl -s -H \"Authorization:
Bearer \$TOK\" 'https://$KV.vault.azure.net/secrets/db-password?api-
version=7.4' | jq -r '.value|length')"
  fi
}

phase_load() { say "Starting 6 minutes of CPU load on the k8s server";
remote k8s "nohup timeout 360 sh -c 'while :; do :; done' >/dev/null
2>&1 &"; ok "Load started. Watch Monitor > Alerts and your mailbox (5 to
10 minutes)."; }

phase_verify() {
  local pass=0 total=0 out
  chk() { total=$((total+1)); if [ "$2" = "$3" ]; then pass=$((pass+1));
ok "$1"; else warn "FAIL: $1 (got '$2', expected '$3')"; fi; }
  out=$(az group show -n "$RG" --query tags.project -o tsv 2>/dev/null);
chk "Resource group exists and is tagged" "$out" "capstone"
  out=$(az network nsg rule list -g "$RG" --nsg-name "$NSG" --query
"[?access=='Allow' && direction=='Inbound' && (sourceAddressPrefix=='*'
|| sourceAddressPrefix=='Internet')] | length(@)" -o tsv 2>/dev/null)
  chk "No inbound rule is open to the whole internet (and no SSH rule)"
"$out" "0"
  for r in ci k8s; do
```

```
    out=$(az vm get-instance-view -g "$RG" -n "$(az_vmname $r)" --query
"instanceView.statuses[?starts_with(code,'PowerState')].displayStatus |
[0]" -o tsv 2>/dev/null)
    chk "VM $r is running" "$out" "VM running"
    out=$(remote $r 'test -f /var/lib/capstone-ready && echo READY'
2>/dev/null | grep -o READY | head -n1); chk "Bootstrap finished on $r"
"$out" "READY"
  done
  out=$(remote ci 'curl -s localhost:5000/v2/capstone-app/tags/list | jq
-r ".tags|sort|join(\",\")"' | tr -d '\r' | grep -Eo '[0-9][0-9,.]*' |
head -n1)
  case "$out" in 1.0,2.0*) out=OK;; *) out="tags=$out";; esac; chk
"Registry holds image tags 1.0 and 2.0" "$out" "OK"
  out=$(remote ci 'curl -s -o /dev/null -w "%{http_code}"
localhost:8080/login' | grep -Eo '^[0-9]{3}|[0-9]{3}$' | head -n1); chk
"Jenkins answers HTTP 200" "$out" "200"
  out=$(remote k8s 'kubectl get nodes --no-headers | grep -c " Ready"' |
grep -Eo '^[0-9]+' | head -n1); chk "K3s node is Ready" "$out" "1"
  out=$(remote k8s 'curl -s localhost:30080/' | grep -o "token=$TOKEN" |
head -n1); chk "App answers with your token on the NodePort" "$out"
"token=$TOKEN"
  out=$(remote ci "TOK=\$(curl -s -H Metadata:true
'http://169.254.169.254/metadata/identity/oauth2/token?api-version=2018-
02-01&resource=https%3A%2F%2Fvault.azure.net' | jq -r .access_token);
curl -s -H \"Authorization: Bearer \$TOK\"
'https://$KV.vault.azure.net/secrets/db-password?api-version=7.4' | jq -
r '.value|length'" | grep -Eo '^[0-9]+' | head -n1)
  [ "${out:-0}" -ge 8 ] 2>/dev/null && out=OK || out="no-read"; chk "VM
managed identity reads the Key Vault secret" "$out" "OK"
  out=$(az monitor metrics alert show -g "$RG" -n "alert-cpu-$P" --query
name -o tsv 2>/dev/null);  chk "CPU alert rule exists" "$out" "alert-
cpu-$P"
  out=$(remote k8s 'ansible-playbook -c local -i localhost, -e
install_docker=false /opt/capstone/ansible/baseline.yml' | grep -o
'changed=[0-9]*' | tail -n1); chk "Ansible re-run is idempotent
(changed=0)" "$out" "changed=0"
  echo; echo "SCORE: $pass / $total      Token: $TOKEN      Student:
$STUDENT_ID      $(date '+%F %T %Z')"
  echo "Take a screenshot of this screen for your evidence folder."
}

main() {
  local cmd="${1:-help}"; shift || true
  [ "$cmd" = help ] && { usage; exit 0; }
  need az; init_workspace; set_names
  case "$cmd" in
    all)      for p in check network compute wait identity monitor
status; do phase_run "$p" "phase_$p" || exit 1; done ;;
    check|network|compute|wait|identity|monitor|status|load|verify)
phase_run "$cmd" "phase_$cmd" ;;
    export)   phase_run export phase_export ;;
    secret)   phase_secret "${1:-read}" ;;
    report)   phase_report ;;
```

```
    run)       remote "${1:?role ci|k8s}" "${2:?command}" ;;
    kube)      remote k8s "kubectl $(printf '%q ' "$@")" ;;
    ansible)   phase_ansible "${1:?role ci|k8s}" ;;
    *)         usage; exit 1 ;;
  esac
}
main "$@"
```

Appendix B: capstone-aws.sh (AWS CloudShell)

Upload to AWS CloudShell. Also delivered as a separate file.

```
#!/usr/bin/env bash
# ================================================================================
# capstone-aws.sh  -  DevOps Bootcamp Capstone (AZ-400 + AZ-104 concepts), AWS edition
# Run in AWS CloudShell. Free-tier instance types. One script, many phases.
# Cleanup: cleanup-aws.sh   (always run it before you leave)
# ================================================================================
# ============================== common helpers ==============================
set -uo pipefail
c_g=$'\033[1;32m'; c_y=$'\033[1;33m'; c_r=$'\033[1;31m'; c_p=$'\033[1;35m'; c_0=$'\033[0m'
say()  { echo "${c_p}==>${c_0} $*"; }
ok()   { echo "${c_g}[ OK ]${c_0} $*"; }
warn() { echo "${c_y}[WARN]${c_0} $*"; }
die()  { echo "${c_r}[FAIL]${c_0} $*" >&2; exit 1; }
need() { command -v "$1" >/dev/null 2>&1 || die "Missing tool: $1"; }

STUDENT_ID="${STUDENT_ID:-}"     # 3-8 lowercase letters/digits, e.g. amk07
MY_IP="${MY_IP:-}"               # your PUBLIC IP as seen by your browser (https://ifconfig.me)
EMAIL="${EMAIL:-}"               # where the CPU alert e-mail is sent
REG_IP="10.50.1.10"              # private IP of the CI server (Jenkins + registry)
K8S_IP="10.50.1.11"              # private IP of the Kubernetes (K3s) server

save() { # save KEY VALUE  -> persists to the state file
 { grep -v "^$1=" "$STATE" 2>/dev/null || true; echo "$1=\"$2\""; } > "$STATE.tmp" && mv
"$STATE.tmp" "$STATE"
}

init_workspace() {
 [[ "$STUDENT_ID" =~ ^[a-z0-9]{3,8}$ ]] || die "Set STUDENT_ID first (3-8 lowercase letters/digits),
e.g.: export STUDENT_ID=amk07"
 P="cap-$STUDENT_ID"
 WORK="$HOME/capstone-$CLOUD-$STUDENT_ID"; mkdir -p "$WORK"
 STATE="$WORK/state.env"; TIMELINE="$WORK/timeline.log"; touch "$STATE" "$TIMELINE"
 # shellcheck disable=SC1090
 . "$STATE"
 if [ -z "${TOKEN:-}" ]; then
   TOKEN=$(printf '%s' "$STUDENT_ID-$(date +%s)" | sha256sum | cut -c1-6); save TOKEN "$TOKEN"
 fi
}

# Every phase is timed: this feeds the value-stream report (Day 1 skill).
phase_run() { # phase_run NAME FUNCTION
 local n=$1 f=$2 s e rc; s=$(date +%s); say "PHASE $n starting"
 "$f"; rc=$?; e=$(date +%s); echo "$n $s $e $rc" >> "$TIMELINE"
 [ $rc -eq 0 ] && ok "PHASE $n finished in $((e-s))s" || warn "PHASE $n ended with code $rc"
 return $rc
}

phase_report() {
 say "Value-stream report for $STUDENT_ID (Day 1: lead time, process time, wait time)"
```

```
 awk '
  { n[NR]=$1; s[NR]=$2; e[NR]=$3; pt+=($3-$2); if(NR==1||$2<first) first=$2; if($3>last) last=$3 }
  END {
    if (NR==0) { print "no phases recorded yet"; exit }
    printf "%-12s %8s\n", "phase", "minutes";
    for (i=1;i<=NR;i++) printf "%-12s %8.1f\n", n[i], (e[i]-s[i])/60;
    lt=last-first; wt=lt-pt;
    printf "\nProcess time (script working) : %6.1f min\n", pt/60;
    printf "Wait time (you + the cloud)   : %6.1f min\n", wt/60;
    printf "Lead time (first to last)     : %6.1f min\n", lt/60;
    if (lt>0) printf "Flow efficiency             : %6.1f %%\n", 100*pt/lt;
  }' "$TIMELINE"
 echo "Your token: $TOKEN   Student: $STUDENT_ID   Time: $(date '+%F %T %Z')"
}

# -------------------------- the app bundle --------------------------------
# Everything the servers need (app, tests, Dockerfile, Jenkinsfile, k8s YAML,
# Ansible playbook, bootstrap) is generated here and shipped inside user-data.
make_bundle() {
 local B="$WORK/bundle"; rm -rf "$B"; mkdir -p "$B/app" "$B/tests" "$B/k8s" "$B/ansible"
 : > "$B/app/__init__.py"
 cat > "$B/app/main.py" <<'PY'
import os
import socket
from flask import Flask

app = Flask(__name__)
VERSION = os.environ.get("APP_VERSION", "dev")
OWNER = os.environ.get("OWNER", "student")
TOKEN = os.environ.get("TOKEN", "none")


@app.route("/")
def home():
   return f"Capstone app v{VERSION} | owner={OWNER} | token={TOKEN}\n"


@app.route("/health")
def health():
   return "ok"


@app.route("/host")
def host():
   return socket.gethostname() + "\n"
PY
 cat > "$B/tests/test_app.py" <<'PY'
from app.main import app


def test_home():
   assert b"Capstone app" in app.test_client().get("/").data
```

```
def test_health():
   assert app.test_client().get("/health").data == b"ok"
PY
 printf 'flask==3.0.3\ngunicorn==22.0.0\npytest==8.3.2\n' > "$B/requirements.txt"
 printf 'flask==3.0.3\ngunicorn==22.0.0\n' > "$B/requirements-runtime.txt"
 printf
'.git\n.venv\n__pycache__\n.pytest_cache\ntests\nk8s\nansible\nbootstrap.sh\nJenkinsfile\n' >
"$B/.dockerignore"
 cat > "$B/Dockerfile" <<'DF'
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements-runtime.txt .
RUN pip install --no-cache-dir -r requirements-runtime.txt
COPY app ./app
ARG APP_VERSION=1.0
ENV APP_VERSION=$APP_VERSION
RUN useradd --create-home --uid 10001 appuser
USER appuser
EXPOSE 5000
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "app.main:app"]
DF
 cat > "$B/Jenkinsfile" <<'JF'
pipeline {
 agent any
 options { timestamps(); timeout(time: 15, unit: 'MINUTES') }
 environment { REGISTRY = 'localhost:5000'; IMAGE = 'capstone-app' }
 stages {
  stage('Checkout') { steps { checkout scm } }
  stage('Test') {
   steps {
    sh '''
     python3 -m venv .venv
     .venv/bin/pip install -q -r requirements.txt
     mkdir -p reports
     .venv/bin/python -m pytest -q --junitxml=reports/junit.xml
    '''
   }
   post { always { junit 'reports/junit.xml' } }
  }
  stage('Build image') {
   steps { sh 'docker build -t $REGISTRY/$IMAGE:$BUILD_NUMBER --build-arg
APP_VERSION=$BUILD_NUMBER .' }
  }
  stage('Scan image') {
   // --exit-code 0 only reports. Change to 1 to FAIL the build on CRITICAL findings.
   steps { sh 'docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy:latest
image --severity CRITICAL --exit-code 0 $REGISTRY/$IMAGE:$BUILD_NUMBER' }
  }
  stage('Approval') {
   steps { timeout(time: 10, unit: 'MINUTES') { input message: 'Publish this build to the registry?' } }
  }
```

```
  stage('Publish') { steps { sh 'docker push $REGISTRY/$IMAGE:$BUILD_NUMBER' } }
 }
 post { failure { echo 'capstone-app: build FAILED - open Console Output and Test Result' } }
}
JF
 cat > "$B/k8s/deployment.yaml" <<'KY'
apiVersion: apps/v1
kind: Deployment
metadata:
 name: capstone-app
spec:
 replicas: 2
 selector:
  matchLabels:
   app: capstone-app
 strategy:
  type: RollingUpdate
  rollingUpdate:
   maxSurge: 1
   maxUnavailable: 0
 template:
  metadata:
   labels:
    app: capstone-app
  spec:
   containers:
    - name: app
      image: REGISTRY_IP:5000/capstone-app:1.0
      ports:
       - containerPort: 5000
      env:
       - name: OWNER
         value: "OWNER_ID"
       - name: TOKEN
         value: "TOKEN_VALUE"
      readinessProbe:
       httpGet: { path: /health, port: 5000 }
       initialDelaySeconds: 3
       periodSeconds: 5
      livenessProbe:
       httpGet: { path: /health, port: 5000 }
       initialDelaySeconds: 10
       periodSeconds: 10
      resources:
       requests: { cpu: 50m, memory: 48Mi }
       limits: { cpu: 250m, memory: 128Mi }
---
apiVersion: v1
kind: Service
metadata:
 name: capstone-app
spec:
 type: NodePort
```

```
 selector:
  app: capstone-app
 ports:
  - port: 80
    targetPort: 5000
    nodePort: 30080
KY
 cat > "$B/ansible/baseline.yml" <<'AY'
---
- name: Capstone baseline (must be idempotent)
 hosts: localhost
 connection: local
 become: true
 vars:
  swap_mb: 2048
  install_docker: false
 tasks:
  - name: Refresh apt cache (not counted as a change)
    ansible.builtin.apt:
      update_cache: true
      cache_valid_time: 3600
    changed_when: false
  - name: Base tools
    ansible.builtin.apt:
      name: [git, curl, jq, unzip]
      state: present
  - name: CI tools (Docker and Python venv)
    ansible.builtin.apt:
      name: [docker.io, python3-venv]
      state: present
    when: install_docker | bool
  - name: Docker service enabled and started
    ansible.builtin.service:
      name: docker
      state: started
      enabled: true
    when: install_docker | bool
  - name: Create swap file
    ansible.builtin.command: "fallocate -l {{ swap_mb }}M /swapfile"
    args:
      creates: /swapfile
  - name: Swap file permissions
    ansible.builtin.file:
      path: /swapfile
      owner: root
      group: root
      mode: "0600"
  - name: Format swap file
    ansible.builtin.shell: "blkid -t TYPE=swap /swapfile >/dev/null 2>&1 || (mkswap /swapfile && echo
formatted)"
    register: fmt
    changed_when: "'formatted' in fmt.stdout"
  - name: Enable swap
```

```
    ansible.builtin.shell: "swapon --show=NAME --noheadings | grep -qx /swapfile || (swapon
/swapfile && echo enabled)"
    register: son
    changed_when: "'enabled' in son.stdout"
  - name: Persist swap in fstab
    ansible.builtin.lineinfile:
      path: /etc/fstab
      line: "/swapfile none swap sw 0 0"
  - name: Marker file proving Ansible managed this server
    ansible.builtin.copy:
      dest: /etc/capstone-baseline.txt
      content: "managed-by=ansible\nswap_mb={{ swap_mb }}\n"
      mode: "0644"
AY
 cat > "$B/bootstrap.sh" <<'BOOT_EOF'
#!/bin/bash
# usage: bootstrap.sh ROLE REG_IP CLOUD OWNER TOKEN    (runs once, as root, from user-data)
set -euo pipefail
ROLE="$1"; REG_IP="$2"; CLOUD="$3"; OWNER="$4"; TOKEN="$5"
exec >> /var/log/capstone-bootstrap.log 2>&1
set -x
export DEBIAN_FRONTEND=noninteractive
echo "$ROLE" > /etc/capstone-role
apt-get update -y
apt-get install -y ansible-core
DOCKER=false
if [ "$ROLE" = "ci" ]; then DOCKER=true; fi
ansible-playbook -c local -i localhost, -e install_docker=$DOCKER
/opt/capstone/ansible/baseline.yml
if [ "$CLOUD" = "aws" ]; then apt-get install -y awscli || snap install aws-cli --classic; fi

if [ "$ROLE" = "ci" ]; then
 cd /opt/capstone
 docker run -d --name registry --restart=always -p 5000:5000 -v registrydata:/var/lib/registry
registry:2
 sleep 5
 for v in 1.0 2.0; do
  docker build -t "localhost:5000/capstone-app:$v" --build-arg "APP_VERSION=$v" .
  docker push "localhost:5000/capstone-app:$v"
 done
 apt-get install -y openjdk-21-jre fontconfig
 install -d -m 0755 /etc/apt/keyrings
 curl -fsSL -o /etc/apt/keyrings/jenkins-keyring.asc https://pkg.jenkins.io/debian-stable/jenkins.io-
2026.key
 echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable
binary/" > /etc/apt/sources.list.d/jenkins.list
 apt-get update -y
 apt-get install -y jenkins
 usermod -aG docker jenkins
 mkdir -p /etc/systemd/system/jenkins.service.d
 printf '[Service]\nEnvironment="JAVA_OPTS=-Djava.awt.headless=true -Xmx512m"\n' >
/etc/systemd/system/jenkins.service.d/override.conf
 systemctl daemon-reload
```

```
 systemctl enable jenkins
 systemctl restart jenkins
fi

if [ "$ROLE" = "k8s" ]; then
 mkdir -p /etc/rancher/k3s
 cat > /etc/rancher/k3s/registries.yaml <<REG_EOF
mirrors:
 "$REG_IP:5000":
  endpoint:
   - "http://$REG_IP:5000"
REG_EOF
 sed -i "s/REGISTRY_IP/$REG_IP/; s/OWNER_ID/$OWNER/; s/TOKEN_VALUE/$TOKEN/"
/opt/capstone/k8s/deployment.yaml
 curl -sfL https://get.k3s.io | INSTALL_K3S_EXEC="server --disable traefik" sh -
 for i in $(seq 1 60); do k3s kubectl get nodes 2>/dev/null | grep -q ' Ready' && break; sleep 5; done
fi
touch /var/lib/capstone-ready
BOOT_EOF
 chmod +x "$B/bootstrap.sh"
 tar czf "$WORK/bundle.tgz" -C "$B" .
 base64 -w 76 "$WORK/bundle.tgz" > "$WORK/bundle.b64"
}

make_userdata() { # make_userdata ROLE OUTFILE   (user-data = a shell script that unpacks the
bundle)
 local role=$1 out=$2
 {
  echo '#!/bin/bash'
  echo 'mkdir -p /opt/capstone'
  echo "cat > /tmp/bundle.b64 <<'B64'"
  cat "$WORK/bundle.b64"
  echo 'B64'
  echo 'base64 -d /tmp/bundle.b64 | tar xz -C /opt/capstone'
  echo "bash /opt/capstone/bootstrap.sh $role $REG_IP $CLOUD $STUDENT_ID $TOKEN"
 } > "$out"
 local size; size=$(stat -c %s "$out")
 [ "$size" -lt "$USERDATA_MAX" ] || die "user-data is $size bytes (limit $USERDATA_MAX)"
 ok "user-data for $role: $size bytes"
}


phase_export() { # writes a Git-ready copy of the app (Mission 1: you push it to YOUR GitHub)
 make_bundle
 local dst="$HOME/capstone-app-$STUDENT_ID"; mkdir -p "$dst"; cp -a "$WORK/bundle/." "$dst/"
 printf '.venv/\n__pycache__/\n.pytest_cache/\n*.pyc\n' > "$dst/.gitignore"
 printf '# capstone-app (%s)\n\nDemo project from the DevOps bootcamp capstone. Not client
work.\n' "$STUDENT_ID" > "$dst/README.md"
 ok "Git-ready folder: $dst"
}

need_my_ip() {
```

```
 [[ "$MY_IP" =~ ^([0-9]{1,3}\.){3}[0-9]{1,3}$ ]] || die "Set MY_IP to your public IP (open
https://ifconfig.me in YOUR browser), e.g.: export MY_IP=203.0.113.7"
 [ "$MY_IP" != "0.0.0.0" ] || die "MY_IP must be your own address, never 0.0.0.0"
}

# ============================== AWS specifics ==============================
CLOUD=aws
REGION="${REGION:-ap-south-1}"          # keep the console and this script in the same region
export AWS_DEFAULT_REGION="$REGION" AWS_PAGER=""
INSTANCE_TYPE="${INSTANCE_TYPE:-t3.micro}" # must be Free-tier eligible in YOUR account
USERDATA_MAX=15500                      # EC2 user-data limit is 16 KB

usage() {
cat <<USAGE
Capstone (AWS)  -  usage:  ./capstone-aws.sh <phase> [args]

 Set these first:  export STUDENT_ID=amk07  MY_IP=<your public IP>  EMAIL=<you@mail>
 Optional:       export REGION=ap-south-1  INSTANCE_TYPE=t3.micro

 all       check, network, identity, compute, wait, monitor, status (about 15 min)
 check     login, region, instance type and input checks (nothing is created)
 network   VPC, subnet, internet gateway, route table, security group (least privilege)
 identity  IAM role + instance profile (SSM access), SecureString parameter
 compute   two Ubuntu 24.04 instances (ci = Jenkins + registry, k8s = K3s), Ansible baseline
 wait      wait until both servers finish their bootstrap
 monitor   SNS topic + CloudWatch CPU alarm
 status    URLs and where to click in the console
 run   ci|k8s "<command>"    run a shell command on a server (Systems Manager, no SSH)
 kube  <kubectl arguments>   run kubectl on the k8s server
 ansible ci|k8s            re-run the Ansible baseline and show the recap
 load                    6 minutes of CPU load on the k8s server (alarm test)
 verify    automatic check of the whole capstone (prints your score and token)
 export    write a Git-ready copy of the app to ~/capstone-app-<id> (Mission 1)
 secret [read|write]       read the secret with the server identity, or try to write it (must fail)
 report    value-stream report (lead time, flow efficiency)
USAGE
}

tspec() { echo
"ResourceType=$1,Tags=[{Key=Name,Value=$2},{Key=project,Value=capstone},{Key=owner,Value=$
STUDENT_ID}]"; }

find_instance() { # find_instance ROLE -> instance id (running/pending/stopped)
 aws ec2 describe-instances --filters "Name=tag:owner,Values=$STUDENT_ID"
"Name=tag:role,Values=$1" \
   "Name=instance-state-name,Values=pending,running,stopping,stopped" --query
'Reservations[].Instances[].InstanceId | [0]' --output text 2>/dev/null
}

remote() { # remote ROLE "command"  (Systems Manager Run Command: no open SSH port
required)
 local id cid st i
```

```
 id=$(find_instance "$1"); [ -n "$id" ] && [ "$id" != None ] || die "No $1 instance found. Run compute
first."
 cid=$(aws ssm send-command --instance-ids "$id" --document-name AWS-RunShellScript \
     --parameters "$(jq -n --arg c "export KUBECONFIG=/etc/rancher/k3s/k3s.yaml; $2"
'{commands:[$c]}')" \
     --query Command.CommandId --output text) || return 1
 for i in $(seq 1 100); do
   st=$(aws ssm get-command-invocation --command-id "$cid" --instance-id "$id" --query Status --
output text 2>/dev/null)
   case "$st" in Success|Failed|Cancelled|TimedOut) break;; esac; sleep 3
 done
 aws ssm get-command-invocation --command-id "$cid" --instance-id "$id" --query
'[StandardOutputContent,StandardErrorContent]' --output text
}

set_names() { ROLE_NAME="$P-ec2-role"; PROFILE="$P-ec2-profile";
PARAM="/capstone/$STUDENT_ID/db-password"; }

phase_check() {
 need aws; need jq; need curl
 aws sts get-caller-identity >/dev/null 2>&1 || die "Not signed in. Open AWS CloudShell from the
console toolbar."
 need_my_ip
 local elig; elig=$(aws ec2 describe-instance-types --filters Name=free-tier-eligible,Values=true \
     --query 'InstanceTypes[].InstanceType' --output text 2>/dev/null)
 if ! echo " $elig " | grep -q " $INSTANCE_TYPE "; then
   [ "${CONFIRM_PAID:-no}" = yes ] || die "$INSTANCE_TYPE is not free-tier eligible for your account.
Eligible types here: $elig . Pick one (export INSTANCE_TYPE=...) or accept the cost (export
CONFIRM_PAID=yes)."
 fi
 ok "Account $(aws sts get-caller-identity --query Account --output text)  Region: $REGION  Type:
$INSTANCE_TYPE"
 warn "Check Billing > Free Tier before you start. Public IPv4 addresses and EBS volumes can cost
cents per hour."
}

phase_network() {
 need_my_ip
 local vpc igw sub rt sg
 vpc=$(aws ec2 create-vpc --cidr-block 10.50.0.0/16 --tag-specifications "$(tspec vpc "$P-vpc")" --
query Vpc.VpcId --output text) || die "VPC failed"
 aws ec2 modify-vpc-attribute --vpc-id "$vpc" --enable-dns-hostnames '{"Value":true}'
 igw=$(aws ec2 create-internet-gateway --tag-specifications "$(tspec internet-gateway "$P-igw")" --
query InternetGateway.InternetGatewayId --output text)
 aws ec2 attach-internet-gateway --internet-gateway-id "$igw" --vpc-id "$vpc"
 sub=$(aws ec2 create-subnet --vpc-id "$vpc" --cidr-block 10.50.1.0/24 --tag-specifications "$(tspec
subnet "$P-subnet")" --query Subnet.SubnetId --output text)
 aws ec2 modify-subnet-attribute --subnet-id "$sub" --map-public-ip-on-launch
 rt=$(aws ec2 create-route-table --vpc-id "$vpc" --tag-specifications "$(tspec route-table "$P-rt")" --
query RouteTable.RouteTableId --output text)
 aws ec2 create-route --route-table-id "$rt" --destination-cidr-block 0.0.0.0/0 --gateway-id
"$igw" >/dev/null
 aws ec2 associate-route-table --route-table-id "$rt" --subnet-id "$sub" >/dev/null
```

```
 sg=$(aws ec2 create-security-group --group-name "$P-sg" --description "capstone least privilege" --
vpc-id "$vpc" \
      --tag-specifications "$(tspec security-group "$P-sg")" --query GroupId --output text)
 aws ec2 authorize-security-group-ingress --group-id "$sg" --ip-permissions \
   "IpProtocol=tcp,FromPort=8080,ToPort=8080,IpRanges=[{CidrIp=$MY_IP/32,Description=jenkins-
from-student}]" \
   "IpProtocol=tcp,FromPort=30080,ToPort=30080,IpRanges=[{CidrIp=$MY_IP/32,Description=app-
from-student}]" \

"IpProtocol=tcp,FromPort=5000,ToPort=5000,IpRanges=[{CidrIp=10.50.1.0/24,Description=registry-
from-subnet}]" >/dev/null
 save VPC "$vpc"; save SUBNET "$sub"; save SG "$sg"
 ok "VPC $vpc, subnet $sub, security group $sg. There is NO SSH rule: servers are managed with
Systems Manager."
}

phase_identity() {
 local trust="$WORK/trust.json" pol="$WORK/param-policy.json"
 cat > "$trust" <<'JSON'
{"Version":"2012-10-
17","Statement":[{"Effect":"Allow","Principal":{"Service":"ec2.amazonaws.com"},"Action":"sts:Assu
meRole"}]}
JSON
 cat > "$pol" <<JSON
{"Version":"2012-10-
17","Statement":[{"Effect":"Allow","Action":["ssm:GetParameter"],"Resource":"arn:aws:ssm:$REGIO
N:*:parameter/capstone/$STUDENT_ID/*"}]}
JSON
 aws iam create-role --role-name "$ROLE_NAME" --assume-role-policy-document "file://$trust" --
tags Key=project,Value=capstone >/dev/null \
   && ok "IAM role $ROLE_NAME"
 aws iam attach-role-policy --role-name "$ROLE_NAME" --policy-arn
arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore
 aws iam put-role-policy --role-name "$ROLE_NAME" --policy-name read-own-params --policy-
document "file://$pol"
 aws iam create-instance-profile --instance-profile-name "$PROFILE" >/dev/null
 aws iam add-role-to-instance-profile --instance-profile-name "$PROFILE" --role-name
"$ROLE_NAME"
 aws ssm get-parameter --name "$PARAM" >/dev/null 2>&1 || \
   aws ssm put-parameter --name "$PARAM" --type SecureString --value "$(openssl rand -base64
12)" --tags Key=project,Value=capstone >/dev/null
 ok "SecureString parameter $PARAM (role may read ONLY /capstone/$STUDENT_ID/*)"
 say "Waiting 15 s for IAM to propagate ..."; sleep 15
}

phase_compute() {
 make_bundle
 make_userdata ci "$WORK/ud-ci.sh"; make_userdata k8s "$WORK/ud-k8s.sh"
 local ami role ip id try extra=()
 ami=$(aws ssm get-parameter --name
/aws/service/canonical/ubuntu/server/24.04/stable/current/amd64/hvm/ebs-gp3/ami-id \
      --query Parameter.Value --output text) || die "Cannot find the Ubuntu 24.04 AMI"
```

```
 case "$INSTANCE_TYPE" in t2.*|t3.*) extra=(--credit-specification CpuCredits=standard);; esac   # no surprise CPU-credit charges
 for role in ci k8s; do
  ip=$REG_IP; [ "$role" = k8s ] && ip=$K8S_IP
  for try in 1 2 3 4 5 6; do
   id=$(aws ec2 run-instances --image-id "$ami" --instance-type "$INSTANCE_TYPE" --subnet-id "$SUBNET" --security-group-ids "$SG" \
     --private-ip-address "$ip" --iam-instance-profile "Name=$PROFILE" --user-data "file://$WORK/ud-$role.sh" \
     --metadata-options HttpTokens=required,HttpEndpoint=enabled \
     --block-device-mappings 'DeviceName=/dev/sda1,Ebs={VolumeSize=20,VolumeType=gp3,DeleteOnTermination=true}' \
     "${extra[@]}" \
     --tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=$P-$role},{Key=project,Value=capstone},{Key=owner,Value=$STUDENT_ID},{Key=role,Value=$role}]" "$(tspec volume "$P-$role-disk")" \
     --query 'Instances[0].InstanceId' --output text 2>/dev/null) && break
   say "instance profile not ready yet, retrying ($try/6) ..."; sleep 10
  done
  [ -n "${id:-}" ] || die "run-instances failed for $role"
  save "INST_$role" "$id"; ok "Instance $role = $id"
 done
 aws ec2 wait instance-running --instance-ids "$(find_instance ci)" "$(find_instance k8s)"
 ok "Both instances are running. They now install software on their own (about 8 to 12 minutes). Use: ./capstone-aws.sh wait"
}

phase_wait() {
 local role n out id
 for role in ci k8s; do
  id=$(find_instance $role); n=0
  while [ $n -lt 40 ]; do
   [ "$(aws ssm describe-instance-information --filters "Key=InstanceIds,Values=$id" --query 'InstanceInformationList[0].PingStatus' --output text 2>/dev/null)" = Online ] && break
   n=$((n+1)); say "$role: waiting for the Systems Manager agent ($n/40) ..."; sleep 15
  done
  n=0
  while [ $n -lt 40 ]; do
   out=$(remote $role 'test -f /var/lib/capstone-ready && echo READY || echo WAIT' 2>/dev/null | grep -Eo 'READY|WAIT' | head -n1)
   [ "$out" = READY ] && { ok "$role server is ready"; break; }
   n=$((n+1)); say "$role still installing ($n/40) ..."; sleep 15
  done
  [ "$out" = READY ] || die "$role not ready. Read the log: ./capstone-aws.sh run $role 'tail -n 40 /var/log/capstone-bootstrap.log'"
 done
}

phase_monitor() {
 [ -n "$EMAIL" ] || die "Set EMAIL first: export EMAIL=you@example.com"
 local topic; topic=$(aws sns create-topic --name "$P-alerts" --tags Key=project,Value=capstone --query TopicArn --output text)
 aws sns subscribe --topic-arn "$topic" --protocol email --notification-endpoint "$EMAIL" >/dev/null
```

```
 ok "SNS topic $P-alerts. OPEN YOUR MAILBOX and click Confirm subscription."
 aws cloudwatch put-metric-alarm --alarm-name "$P-cpu-k8s" --alarm-description "Capstone: k8s server CPU above 70 percent" \
   --namespace AWS/EC2 --metric-name CPUUtilization --dimensions "Name=InstanceId,Value=$(find_instance k8s)" \
   --statistic Average --period 300 --evaluation-periods 1 --threshold 70 --comparison-operator GreaterThanThreshold \
   --alarm-actions "$topic" --tags Key=project,Value=capstone && ok "CloudWatch alarm $P-cpu-k8s (CPU > 70 percent over 5 minutes)"
}

phase_status() {
 local ci k8
 ci=$(aws ec2 describe-instances --instance-ids "$(find_instance ci)" --query 'Reservations[0].Instances[0].PublicIpAddress' --output text)
 k8=$(aws ec2 describe-instances --instance-ids "$(find_instance k8s)" --query 'Reservations[0].Instances[0].PublicIpAddress' --output text)
 echo; echo "Jenkins (from YOUR browser/IP only) : http://$ci:8080"
 echo "App on Kubernetes (NodePort)       : http://$k8:30080"
 echo "Console: EC2 > Instances (tag owner=$STUDENT_ID)  |  Parameter: $PARAM  |  Token: $TOKEN"
 echo "Jenkins unlock password : ./capstone-aws.sh run ci 'cat /var/lib/jenkins/secrets/initialAdminPassword'"
}

phase_ansible() { remote "$1" "ansible-playbook -c local -i localhost, -e install_docker=$([ "$1" = ci ] && echo true || echo false) /opt/capstone/ansible/baseline.yml" | tail -n 25; }

phase_secret() { # secret [read|write]  - uses the instance role, no access key anywhere
 if [ "${1:-read}" = write ]; then
   say "Trying to WRITE a parameter with the read-only role (must be refused)"
   remote ci "aws ssm put-parameter --name /capstone/$STUDENT_ID/not-allowed --value x --type String --region $REGION 2>&1 | tail -n 2"
 else
   say "Reading the secret with the instance role (only the length is shown)"
   remote ci "echo secret length: \$(aws ssm get-parameter --name $PARAM --with-decryption --region $REGION --query 'length(Parameter.Value)' --output text)"
 fi
}

phase_load() { say "Starting 6 minutes of CPU load on the k8s server"; remote k8s "nohup timeout 360 sh -c 'while :; do :; done' >/dev/null 2>&1 &"; ok "Load started. Watch CloudWatch > Alarms and your mailbox (5 to 10 minutes)."; }

phase_verify() {
 local pass=0 total=0 out r
 chk() { total=$((total+1)); if [ "$2" = "$3" ]; then pass=$((pass+1)); ok "$1"; else warn "FAIL: $1 (got '$2', expected '$3')"; fi; }
 out=$(aws ec2 describe-vpcs --filters "Name=tag:owner,Values=$STUDENT_ID" "Name=tag:project,Values=capstone" --query 'length(Vpcs)' --output text 2>/dev/null); chk "VPC exists and is tagged" "$out" "1"
```

```
  out=$(aws ec2 describe-security-groups --group-ids "${SG:-none}" --query
"length(SecurityGroups[0].IpPermissions[?IpRanges[?CidrIp=='0.0.0.0/0']])" --output text
2>/dev/null)
  chk "No inbound rule is open to 0.0.0.0/0 (and no SSH rule)" "$out" "0"
  for r in ci k8s; do
    out=$(aws ec2 describe-instances --instance-ids "$(find_instance $r)" --query
'Reservations[0].Instances[0].State.Name' --output text 2>/dev/null); chk "Instance $r is running"
"$out" "running"
    out=$(aws ec2 describe-instances --instance-ids "$(find_instance $r)" --query
'Reservations[0].Instances[0].MetadataOptions.HttpTokens' --output text 2>/dev/null); chk "Instance
$r enforces IMDSv2" "$out" "required"
    out=$(remote $r 'test -f /var/lib/capstone-ready && echo READY' 2>/dev/null | grep -o READY | head
-n1); chk "Bootstrap finished on $r" "$out" "READY"
  done
  out=$(remote ci 'curl -s localhost:5000/v2/capstone-app/tags/list | jq -r ".tags|sort|join(\",\")"' | grep
-Eo '[0-9][0-9,.]*' | head -n1)
  case "$out" in 1.0,2.0*) out=OK;; *) out="tags=$out";; esac; chk "Registry holds image tags 1.0 and
2.0" "$out" "OK"
  out=$(remote ci 'curl -s -o /dev/null -w "%{http_code}" localhost:8080/login' | grep -Eo '^[0-9]{3}|[0-
9]{3}$' | head -n1); chk "Jenkins answers HTTP 200" "$out" "200"
  out=$(remote k8s 'kubectl get nodes --no-headers | grep -c " Ready"' | grep -Eo '^[0-9]+' | head -n1);
chk "K3s node is Ready" "$out" "1"
  out=$(remote k8s 'curl -s localhost:30080/' | grep -o "token=$TOKEN" | head -n1); chk "App answers
with your token on the NodePort" "$out" "token=$TOKEN"
  out=$(remote ci "aws ssm get-parameter --name $PARAM --with-decryption --region $REGION --
query 'length(Parameter.Value)' --output text" | grep -Eo '^[0-9]+' | head -n1)
  [ "${out:-0}" -ge 8 ] 2>/dev/null && out=OK || out="no-read"; chk "Instance role reads the
SecureString parameter" "$out" "OK"
  out=$(aws cloudwatch describe-alarms --alarm-names "$P-cpu-k8s" --query
'MetricAlarms[0].AlarmName' --output text 2>/dev/null); chk "CPU alarm exists" "$out" "$P-cpu-k8s"
  out=$(remote k8s 'ansible-playbook -c local -i localhost, -e install_docker=false
/opt/capstone/ansible/baseline.yml' | grep -o 'changed=[0-9]*' | tail -n1); chk "Ansible re-run is
idempotent (changed=0)" "$out" "changed=0"
  echo; echo "SCORE: $pass / $total     Token: $TOKEN     Student: $STUDENT_ID     $(date
'+%F %T %Z')"
  echo "Take a screenshot of this screen for your evidence folder."
}

main() {
  local cmd="${1:-help}"; shift || true
  [ "$cmd" = help ] && { usage; exit 0; }
  need aws; init_workspace; set_names
  case "$cmd" in
    all)     for p in check network identity compute wait monitor status; do phase_run "$p" "phase_$p" ||
exit 1; done ;;
    check|network|identity|compute|wait|monitor|status|load|verify) phase_run "$cmd"
"phase_$cmd" ;;
    export)   phase_run export phase_export ;;
    secret)   phase_secret "${1:-read}" ;;
    report)   phase_report ;;
    run)     remote "${1:?role ci|k8s}" "${2:?command}" ;;
    kube)    remote k8s "kubectl $(printf '%q ' "$@")" ;;
    ansible)  phase_ansible "${1:?role ci|k8s}" ;;
```

```
  *)      usage; exit 1 ;;
 esac
}
main "$@"
```

Appendix C: cleanup-azure.sh

Removes every Azure resource of the capstone, including the locked Key Vault, then purges the soft-deleted vault.

```
#!/usr/bin/env bash
# ================================================================================
# cleanup-azure.sh  -  removes EVERYTHING the Azure capstone created
# Run in Azure Cloud Shell (Bash):  export STUDENT_ID=amk07 ; ./cleanup-azure.sh
# Skip the question with:  CONFIRM=yes ./cleanup-azure.sh
# ================================================================================
set -uo pipefail
say() { echo "==> $*"; }; ok() { echo "[ OK ] $*"; }; warn() { echo "[WARN] $*"; }; die() { echo "[FAIL]
$*" >&2; exit 1; }
STUDENT_ID="${STUDENT_ID:-}"
[[ "$STUDENT_ID" =~ ^[a-z0-9]{3,8}$ ]] || die "Set STUDENT_ID first, e.g.: export STUDENT_ID=amk07"
az account show >/dev/null 2>&1 || die "Not signed in to Azure."
P="cap-$STUDENT_ID"; RG="rg-$P"; WORK="$HOME/capstone-azure-$STUDENT_ID"
[ -f "$WORK/state.env" ] && . "$WORK/state.env"
if ! az group exists -n "$RG" | grep -q true; then
 warn "Resource group $RG does not exist (already deleted?). Checking for leftovers only."
else
 say "These resources will be deleted:"; az resource list -g "$RG" --query '[].{name:name,type:type}' -
o table
 if [ "${CONFIRM:-}" != yes ]; then
  read -r -p "Type the resource group name ($RG) to confirm: " ans; [ "$ans" = "$RG" ] || die "Not
confirmed. Nothing was deleted."
 fi
 say "Removing resource locks (a CanNotDelete lock blocks deletion)"
 for id in $(az lock list -g "$RG" --query '[].id' -o tsv); do az lock delete --ids "$id" && ok "lock
removed"; done
 say "Deleting resource group $RG (VMs, disks, public IPs, NSG, VNet, Key Vault, alerts) ..."
 az group delete -n "$RG" --yes --no-wait
 az group wait -n "$RG" --deleted --timeout 1500 2>/dev/null; ok "Resource group deleted"
fi
say "Purging soft-deleted Key Vaults of this student (frees the name, stops retention)"
for kv in $(az keyvault list-deleted --query "[?starts_with(name,'kv${STUDENT_ID}')].name" -o tsv); do
 loc=$(az keyvault list-deleted --query "[?name=='$kv'].properties.location | [0]" -o tsv)
 az keyvault purge --name "$kv" --location "$loc" >/dev/null 2>&1 && ok "purged $kv" || warn "could
not purge $kv (check Key Vault > Manage deleted vaults)"
done
say "Final check"
left=$(az resource list --tag project=capstone --query "[?tags.owner=='$STUDENT_ID'] | length(@)" -o
tsv)
if az group exists -n "$RG" | grep -q false && [ "${left:-0}" = 0 ]; then ok "CLEAN: no resource group and
no tagged resources left for $STUDENT_ID"; else warn "Something is left. Portal > All resources, filter
tag project=capstone."; fi
[ -f "$WORK/timeline.log" ] && cp "$WORK/timeline.log" "$HOME/capstone-timeline-
$STUDENT_ID.log"
rm -rf "$WORK"; ok "Local work folder removed (timeline copy kept in your home folder)"
echo "Still to check by hand: Cost Management + Billing > Cost analysis (should show about 0), and
any Action Group e-mails."
```

```
echo "Cloud Shell's own storage account (rg cloud-shell-storage-*) is NOT part of this lab. Delete it at the end of the course."
```

Appendix D: cleanup-aws.sh

Terminates the instances, deletes alarm, topic, parameter, IAM role and the whole VPC, then verifies nothing is left.

```
#!/usr/bin/env bash
#
================================================================================
=====
# cleanup-aws.sh  -  removes EVERYTHING the AWS capstone created
# Run in AWS CloudShell:  export STUDENT_ID=amk07 ; ./cleanup-aws.sh
# Skip the question with:  CONFIRM=yes ./cleanup-aws.sh      (set REGION
if you did not use ap-south-1)
#
================================================================================
=====
set -uo pipefail
say() { echo "==> $*"; }; ok() { echo "[ OK ] $*"; }; warn() { echo
"[WARN] $*"; }; die() { echo "[FAIL] $*" >&2; exit 1; }
STUDENT_ID="${STUDENT_ID:-}"; REGION="${REGION:-ap-south-1}"
export AWS_DEFAULT_REGION="$REGION" AWS_PAGER=""
[[ "$STUDENT_ID" =~ ^[a-z0-9]{3,8}$ ]] || die "Set STUDENT_ID first,
e.g.: export STUDENT_ID=amk07"
aws sts get-caller-identity >/dev/null 2>&1 || die "Not signed in to
AWS."
P="cap-$STUDENT_ID"; ROLE="$P-ec2-role"; PROFILE="$P-ec2-profile";
PARAM="/capstone/$STUDENT_ID/db-password"; WORK="$HOME/capstone-aws-
$STUDENT_ID"
F=("Name=tag:owner,Values=$STUDENT_ID"
"Name=tag:project,Values=capstone")

IDS=$(aws ec2 describe-instances --filters "${F[@]}" "Name=instance-
state-name,Values=pending,running,stopping,stopped" --query
'Reservations[].Instances[].InstanceId' --output text)
VPCS=$(aws ec2 describe-vpcs --filters "${F[@]}" --query 'Vpcs[].VpcId'
--output text)
say "Region $REGION. Found instances: ${IDS:-none}   VPCs: ${VPCS:-
none}"
if [ "${CONFIRM:-}" != yes ]; then read -r -p "Type the student id
($STUDENT_ID) to confirm deletion: " ans; [ "$ans" = "$STUDENT_ID" ] ||
die "Not confirmed. Nothing was deleted."; fi

if [ -n "$IDS" ]; then
  say "Terminating instances (their EBS disks are deleted with
them) ..."
  aws ec2 terminate-instances --instance-ids $IDS >/dev/null; aws ec2
wait instance-terminated --instance-ids $IDS && ok "instances
terminated"
fi
say "Alarm, SNS topic, SSM parameter"
aws cloudwatch delete-alarms --alarm-names "$P-cpu-k8s" 2>/dev/null &&
ok "alarm deleted"
```

```
for t in $(aws sns list-topics --query "Topics[?ends_with(TopicArn,':$P-
alerts')].TopicArn" --output text); do aws sns delete-topic --topic-arn
"$t" && ok "topic deleted"; done
aws ssm delete-parameter --name "$PARAM" 2>/dev/null && ok "parameter
deleted"
say "IAM instance profile and role"
aws iam remove-role-from-instance-profile --instance-profile-name
"$PROFILE" --role-name "$ROLE" 2>/dev/null
aws iam delete-instance-profile --instance-profile-name "$PROFILE"
2>/dev/null && ok "instance profile deleted"
aws iam detach-role-policy --role-name "$ROLE" --policy-arn
arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore 2>/dev/null
aws iam delete-role-policy --role-name "$ROLE" --policy-name read-own-
params 2>/dev/null
aws iam delete-role --role-name "$ROLE" 2>/dev/null && ok "role deleted"

for vpc in $VPCS; do
  say "Deleting network $vpc"
  for sg in $(aws ec2 describe-security-groups --filters "Name=vpc-
id,Values=$vpc" --query "SecurityGroups[?GroupName!='default'].GroupId"
--output text); do
    for n in 1 2 3 4 5 6; do aws ec2 delete-security-group --group-id
"$sg" 2>/dev/null && { ok "security group $sg deleted"; break; }; sleep
10; done
  done
  for rt in $(aws ec2 describe-route-tables --filters "Name=vpc-
id,Values=$vpc" --query
"RouteTables[?length(Associations[?Main==\`true\`])==\`0\`].RouteTableId
" --output text); do
    for a in $(aws ec2 describe-route-tables --route-table-ids "$rt" --
query 'RouteTables[0].Associations[].RouteTableAssociationId' --output
text); do aws ec2 disassociate-route-table --association-id "$a"; done
    aws ec2 delete-route-table --route-table-id "$rt" && ok "route table
$rt deleted"
  done
  for sub in $(aws ec2 describe-subnets --filters "Name=vpc-
id,Values=$vpc" --query 'Subnets[].SubnetId' --output text); do aws ec2
delete-subnet --subnet-id "$sub" && ok "subnet $sub deleted"; done
  for igw in $(aws ec2 describe-internet-gateways --filters
"Name=attachment.vpc-id,Values=$vpc" --query
'InternetGateways[].InternetGatewayId' --output text); do
    aws ec2 detach-internet-gateway --internet-gateway-id "$igw" --vpc-
id "$vpc"; aws ec2 delete-internet-gateway --internet-gateway-id "$igw"
&& ok "internet gateway $igw deleted"
  done
  aws ec2 delete-vpc --vpc-id "$vpc" && ok "VPC $vpc deleted"
done

say "Final check"
left_i=$(aws ec2 describe-instances --filters "${F[@]}" "Name=instance-
state-name,Values=pending,running,stopping,stopped" --query
'length(Reservations[].Instances[])' --output text)
```

```
left_v=$(aws ec2 describe-vpcs --filters "${F[@]}" --query
'length(Vpcs)' --output text)
left_vol=$(aws ec2 describe-volumes --filters "${F[@]}" --query
'length(Volumes)' --output text)
left_r=$(aws iam get-role --role-name "$ROLE" >/dev/null 2>&1 && echo 1
|| echo 0)
if [ "$left_i$left_v$left_vol$left_r" = "0000" ]; then ok "CLEAN: no
instances, volumes, VPC or IAM role left for $STUDENT_ID"; else warn
"Left over: instances=$left_i vpcs=$left_v volumes=$left_vol
role=$left_r. Re-run this script, then check the console."; fi
[ -f "$WORK/timeline.log" ] && cp "$WORK/timeline.log" "$HOME/capstone-
timeline-$STUDENT_ID.log"
rm -rf "$WORK"; ok "Local work folder removed (timeline copy kept in
your home folder)"
echo "Still to check by hand: Billing > Free Tier (usage), and the SNS
confirmation e-mail can be ignored."
```